Appunti di Programmazione Funzionale

Alessio Novel

Introduzione....5
Macchine astratte....7
La nozione di macchina astratta e l’interprete....7
Interprete....8
Un esempio di macchina astratta: la macchina hardware....9
Implementazione di un linguaggio....9
Realizzazione di una macchina astratta....10
Implementazione: il caso ideale....10
Implementazione interpretativa pura....11
Implementazione compilativa pura....11
Confronto fra le due....12
Implementazione: il caso reale e la macchina intermedia....12
Gerarchie di macchine astratte....13
I nomi e l’ambiente....13
Nomi e oggetti denotabili....13
Oggetti denotabili....13
Ambiente e blocchi....14
Blocchi....14
Tipi di ambiente....14
Operazioni sull’ambiente....15
Regole di scope....16
Scope statico....16
Scope dinamico....16
La gestione della memoria....17
Gestione statica....17
Gestione dinamica tramite stack....18
RdA per i blocchi in-line....18
RdA per le procedure....18
Gestione della pila....19
Gestione dinamica mediante heap....20
Blocchi di dimensione fissa....20
Blocchi di dimensione variabile....20
Unica lista libera....20
Liste libere multiple....21
Implementazione delle regole di scope....21
Scope statico....21
La catena statica....21
Il display....22
Scope dinamico....22

Lista di associazioni....22
CRT....22
Astrarre sul controllo....23
Sottoprogrammi....23
Astrazione funzionale....23
Passaggio dei parametri....24
Funzioni di ordine superiore....25
Funzioni come parametro....26
Funzioni come risultato....27
Eccezioni....27
Implementare le eccezioni....29
Strutturare i dati....29
Tipi di dato....29
Tipi come supporto all’organizzazione concettuale....29
Tipi per la correttezza....30
Tipi e implementazione....30
Sistemi di tipi....30
Controlli statici e dinamici....31
Tipi scalari....31
Booleani....31
Caratteri....31
Interi....32
Reali....32
Virgola fissa....32
Complessi....32
Void....33
Enumerazioni....33
Intervalli....33
Tipi ordinali....33
Tipi composti....34
Record....34
Record varianti e unioni....34
Array....36
Insiemi....37
Puntatori....37
Tipi ricorsivi....38
Funzioni....38
Equivalenza....39
Equivalenza per nome....39
Equivalenza strutturale....39
Compatibilità e conversione....40
Coercizioni....40
Conversioni esplicite....41
Polimorfismo....41

Overloading....41
Polimorfismo universale parametrico....42
Polimorfismo universale di sottotipo....42
Evitare i dangling reference....42
Tombstone....42
Lucchetti e chiavi....43
Garbage collection....43
Contatore dei riferimenti....44
Mark and sweep....44
Intermezzo: rovesciare i puntatori....45
Mark and compact....46
Copia....46
Astrarre sui dati....46
Tipi di dato astratti....46
Nascondere l’informazione....47
Indipendenza della rappresentazione....48
Moduli....48
Il paradigma logico....48
Deduzione come computazione....48
Sintassi....49
Il linguaggio della logica del prim’ordine....49
Alfabeto....49
Termini....50
Formule....51
I programmi logici....51
Teoria dell’unificazione....51
La variabile logica....52
Sostituzione....52
L’unificatore più generale....53
Un algoritmo di unificazione....53
Il modello computazionale....54
L’universo di Herbrand....54
Interpretazione dichiarativa e procedurale....55
La chiamata di procedura....55
Valutazione di un goal non atomico....55
Teste con termini generici....56
Controllo: il non determinismo....56
Il backtracking in PROLOG....56
Alcuni esempi....56
Estensioni....57
PROLOG....57
Aritmetica....57
Cut....57
Disgiunzione....57

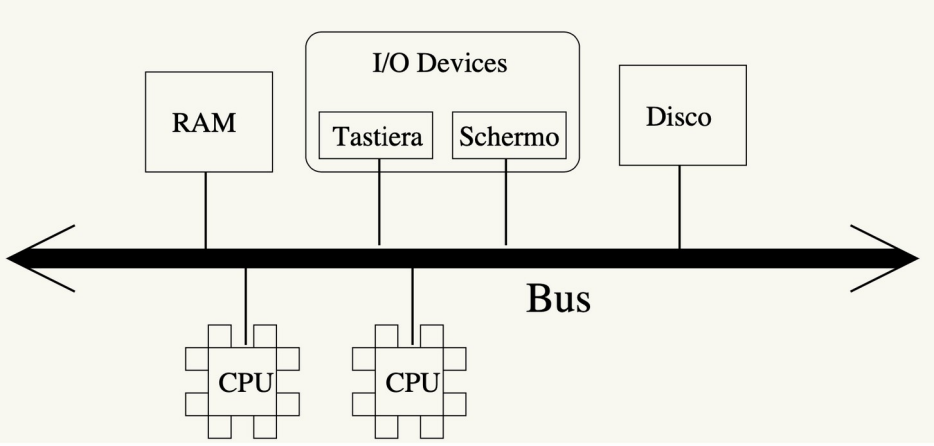
If-then-else....57
Negazione....58
Programmazione logica e basi di dati....58
Programmazione logica con vincoli....58
Pregi e difetti del paradigma logico....58

Introduzione

Nel 1928 viene “creato” Entscheidungsproblem, ovvero la sfida di creare un algoritmo che data una affermazione matematica risponda o “Si” o “No”, in base a se è vera o falsa, ma più avanti si scoprì che i sistemi matematici, seppur coerenti, sono incompleti e indecidibili. Questa scoperta venne fatta tramite il Lambda Calculus
Nel 1932 viene inventato il Lambda Calculus, un modo per analizzare formalmente le funzioni e il loro calcolo, il quale è la base della programmazione funzionale
Nel 1936 venne inventata la macchina di Turing, che riusciva a interpretare dati ed eseguire algoritmi presi da un nastro potenzialmente infinito
I computer capiscono il linguaggio macchina (sequenze di 0 e 1), esistono linguaggi come ASSEMBLY che traducono un linguaggio con una sintassi più semplice per il programmatore in linguaggio macchina
1960, Algol viene implementato con logica stack
COBOL: con l’intento di programmare in un linguaggio quasi parlato, focalizzato su gestione di dati.
Componenti base computer per poter funzionare:

- CPU
    - Esegue le istruzioni macchina
    - Per farlo muove i dati da e verso la memoria principale (RAM)
- RAM
    - Contiene i dati e i programmi (ovvero sequenze di istruzioni macchina)
    - Veloce ma volatile
- Memoria di massa
    - Più lenta ma persistente
- Periferiche I/O

Tutti questi dispositivi sono collegati da un BUS, i canali che collegano i vari componenti, e permette lo scambio di dati tra CPU e RAM
Architettura/schema di Von Neumann



La memoria contiene sia dati che istruzioni
La CPU è composta, oltre che da vari registri, da una parte di controllo (che ottiene e esegue le istruzioni) e una parte operativa, la ALU (Arithmetic Logic Unit) che esegue operazioni

aritmetiche e logiche. Le CPU moderne hanno anche un FPU (Floating Arithmetic Part) per gestire i numeri con la virgola in notazione esponenziale.
Registri non accessibili direttamente dal programmatore:

- Address Register (AR) (per vedere dove sto leggendo o scrivendo)
- Data Register (DR) (sia per scrivere che leggere)

Registri visibili al programmatore:

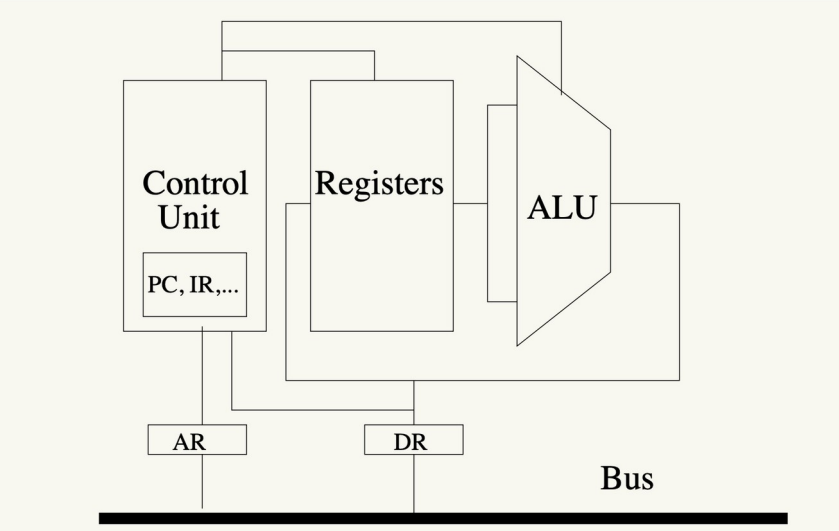
- Program Counter (PC/IP) (indirizzo della prossima istruzione da eseguire)
  Quando si runna un programma ogni volta va a vedere dove è la prossima istruzione nel PC
- Status Register (SR) (un flag per vedere il risultato di un'operazione ecc.)
- Altri registri per data e indirizzi

Execution of instructions

- Read the next instruction to be executed (fetch)
  - Copy the program counter (PC) into the address register (AR)
  - Transfer the data addressed in the AR from RAM to the Data register (DR)
  - Save the DR in an invisibile register
  - Increment the PC
- Decode the instruction
- Execute the decoded instruction
  - If necessary, load data from memory, set the AR, read the DR, etc
  - If necessary, save data in memory, set the AR etc
  - If necessary, modify the PC (jump instructions)

Per accedere alla memoria principale (RAM) bisogna usare il bus, caricando l'indirizzo di memoria nel AR, poi nel caso di scrittura caricare i dati nel DR, specificare l'operazione (write/read), e in caso di lettura i dati verranno scritti nel DR.

Example of a CPU



Questo è un esempio di architettura di CPU anche se al giorno d'oggi sono molto più complesse (ad esempio per il parallelismo e per la pipeline)
L'esecuzione di un programma è un "infinito" ciclo di fetch/decode/load/execute/save e questo ciclo viene fatto finché non si spegne il pc

Le macchine fisiche possono capire solo i programmi scritti nel proprio linguaggio, la CPU ha il ciclo "infinito" implementato nel proprio hardware

Macchine astratte

La nozione di macchina astratta e l'interprete

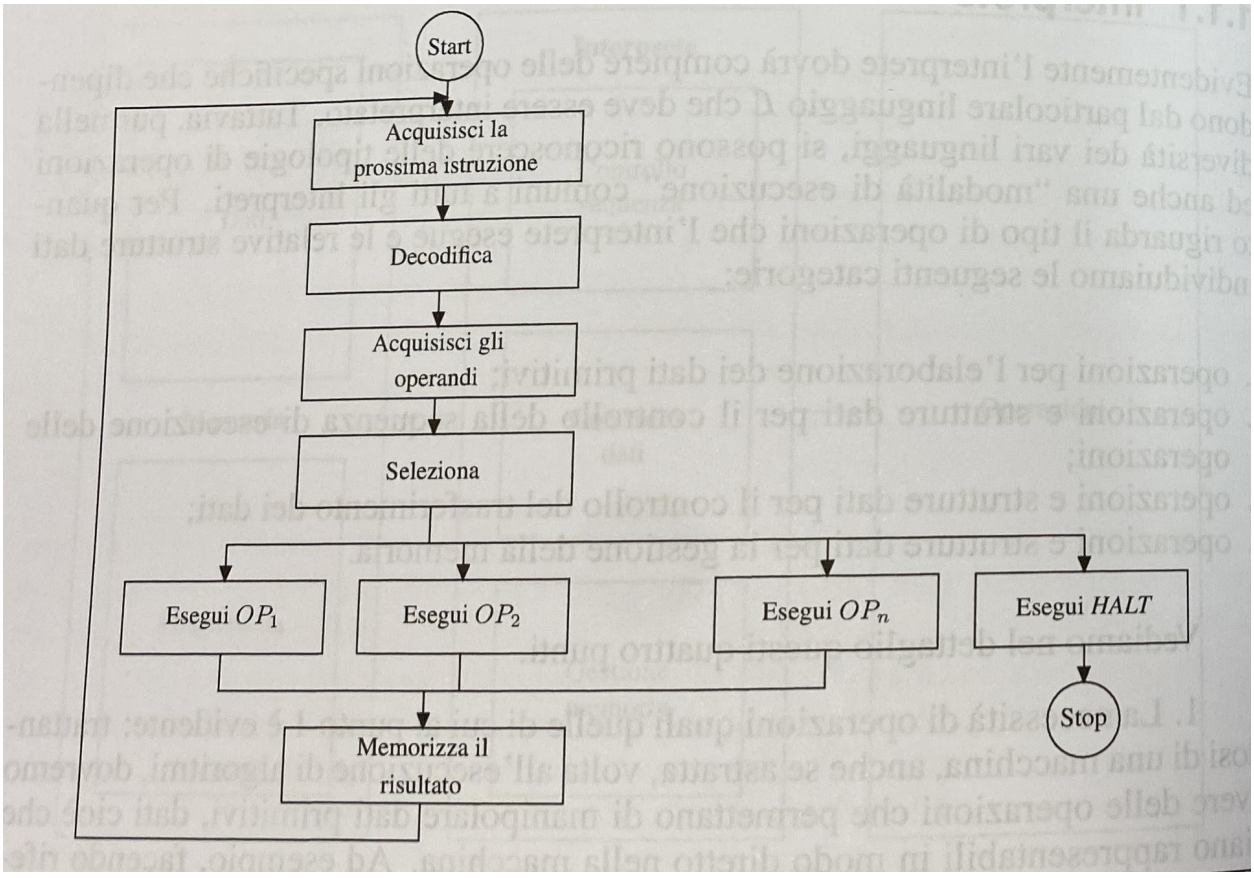
Una macchina fisica (calcolatore) è un dispositivo che permette di eseguire algoritmi, formalizzati per poter essere comprensibili da questa macchina.
Una macchina astratta non è altro che un'astrazione del concetto di calcolatore fisico.
Quindi una macchina astratta avrà un linguaggio L che sarà formalmente definito da una specifica sintassi e da una precisa semantica.
Un programma di L (oppure anche "programma scritto in L") è un insieme finito di istruzioni di L .
Quindi dato un linguaggio di programmazione L, definiamo una macchina astratta per L, e la indichiamo con M_L, un qualsiasi insieme di strutture dati e di algoritmi che permettono di memorizzare ed eseguire programmi in L.
Una macchina astratta per poter funzionare ha bisogno di una **memoria** (per poter immagazzinare i dati, e di un **interprete** (il componente che esegue le istruzioni contenute nei programmi)

Interprete

L'interprete quindi dovrà compiere delle operazioni specifiche che dipendono ovviamente dal linguaggio L che deve essere interpretato.
In genere tutti gli interpreti hanno in comune le seguenti tipologie di operazioni:

1. Operazioni per l'elaborazione dei dati primitivi
   Ad esempio nelle macchine fisiche, che hanno i numeri reali e interi come dati primitivi, esiste la ALU che esegue le operazioni aritmetiche
2. Operazioni e strutture dati per il controllo della sequenza di esecuzione delle operazioni
   Servono per gestire il flusso di esecuzione; la normale esecuzione sequenziale può dover essere modificata in corrispondenza del verificarsi di determinate condizioni.
   Una di queste operazioni potrebbe essere modificare l'indirizzo della prossima istruzione da eseguire
3. Operazioni e strutture dati per il controllo del trasferimento dei dati
   Definiscono in che maniera gli operandi (i dati) devono essere trasferiti dalla memoria all'interprete e viceversa
4. Operazioni e strutture dati per la gestione della memoria
   Riguarda tutte le operazioni per l'allocazione di memoria per i dati e per i programmi

Il ciclo di esecuzione dell'interprete è il seguente:



Data una macchina astratta M_L, il linguaggio L "compreso" dall'interprete di M_L è detto linguaggio macchina di M_L

I programmi scritti nel linguaggio macchina di M_L vengono salvati, come gli altri dati, nelle strutture di memoria della macchina

Un esempio di macchina astratta: la macchina hardware

Il concetto di macchina astratta si può usare per una varietà di sistemi diversi, che vanno dalla macchina hardware al WWW
Prendiamo come esempio una macchina hardware che chiameremo MH_{LH} e il suo linguaggio macchina LH

La memoria è costituita da componenti fisiche e permette di memorizzare i dati tramite l'alfabeto binario. I dati sono divisi in pochi tipi primitivi e a seconda del tipo di dato avremo rappresentazioni fisiche diverse

Il linguaggio LH che la macchina hardware è in grado di eseguire è costituito da istruzioni relativamente semplici.

L'interprete di una macchina hardware in genere è composto da:

1. Operazioni per l'elaborazione dei dati primitivi
   ALU
2. Operazioni e strutture dati per il controllo della sequenza di esecuzione delle operazioni
   Operazioni che operano su PC (program counter)
3. Operazioni e strutture dati per il controllo del trasferimento dei dati
   Le operazioni di questo tipo fanno uso di due registri, l'AR (address register) e DR (data register) e permettono di eseguire varie modalità di indirizzamento.
   In più troviamo le operazioni per accedere e modificare i registri interni della CPU
4. Operazioni e strutture dati per la gestione della memoria
   Cache, allocazione dinamica degli indirizzi, stack etc.

L'interprete di una macchina hardware è realizzato dalla UC (unità di controllo), che permette di eseguire il ciclo fetch-decode-execute visto in precedenza:

1. fetch: preleva l'istruzione il cui indirizzo è contenuto in PC e la mette in un apposito registro
2. decode: l'istruzione prelevata viene decodificata e vengono recuperati gli operandi
3. execute: viene effettivamente eseguita l'operazione primitiva della macchina, ad esempio tramite la ALU

Da notare che anche se la macchina distingue i dati dalle istruzioni, a livello fisico non vi è alcuna distinzione, visto che sono tutte sequenze di bit

Implementazione di un linguaggio

Una macchina M_L può capire solo il **suo** linguaggio macchina (cioè L).
Un linguaggio L invece può essere il linguaggio macchina di un numero infinito di macchine astratte, che differiscono tra loro per il modo in cui l'interprete è realizzato e nelle strutture dati che utilizzano.

Implementare un linguaggio L significa realizzare una macchina astratta che abbia L come linguaggio macchina.

Realizzazione di una macchina astratta

Una qualsiasi macchina M_L dovrà fare prima o poi uso di qualche dispositivo fisico per poter eseguire le istruzioni di L. L'uso di tale dispositivo però può essere esplicito o implicito.
Distinguiamo infatti 3 casi:

- Realizzazione in hardware
  Si tratta di realizzare, mediante dispositivi fisici, una macchina fisica tale che il suo linguaggio coincida con L, realizzando in hardware le strutture dati e gli algoritmi che costituiscono la macchina astratta
  I vantaggi sono che l'esecuzione dei programmi è molto veloce, visto che sono eseguiti direttamente su dispositivi fisici
  Il problema è che i costrutti disponibili in linguaggi ad alto livello sono impossibili da replicare fisicamente mediante circuiti fisici, e anche nei casi in cui è possibile, non è conveniente visto che si userebbero un sacco di componenti e sarebbe impossibile da modificare.
  È per questo motivo che per le implementazioni hardware si usano linguaggi di basso livello
- Simulazione mediante software
  In questa situazione, le implementazione delle strutture dati e degli algoritmi di M_L vengono fatte mediante programmi scritti in un altro linguaggio L', il quale possiamo supporre già implementato. Quindi andremo a simulare le funzionalità di M_L
  In questo caso avremo la massima flessibilità, ma l'esecuzione sarà più lenta in quanto dovrà passare da un livello intermedio, che a sua volta dovrà essere interpretato.
- Simulazione (emulazione) mediante firmware
  Implementare le strutture dati e gli algoritmi di M_L mediante microprogrammi (assembly)
  A livello di prestazioni è una via di mezzo tra le due

Implementazione: il caso ideale

Assumiamo di voler implementare il linguaggio L su una macchina astratta Mo_{Lo} (macchina ospite) che ci permette di usare i costrutti del linguaggio Lo.
A questo punto si hanno due modalità di implementazione per fare in modo che L venga in qualche modo "tradotto" in Lo:

- Implementazione interpretativa pura
- Implementazione compilativa pura

Da adesso in poi L come pedice indicherà che un costrutto si riferisce a quel linguaggio, invece L come apice indicherà che quel costrutto è scritto in quel linguaggio.
Un programma scritto in L può essere visto come una funzione $P^L : D \rightarrow D$ tale che:

$$P^L(input) = output$$

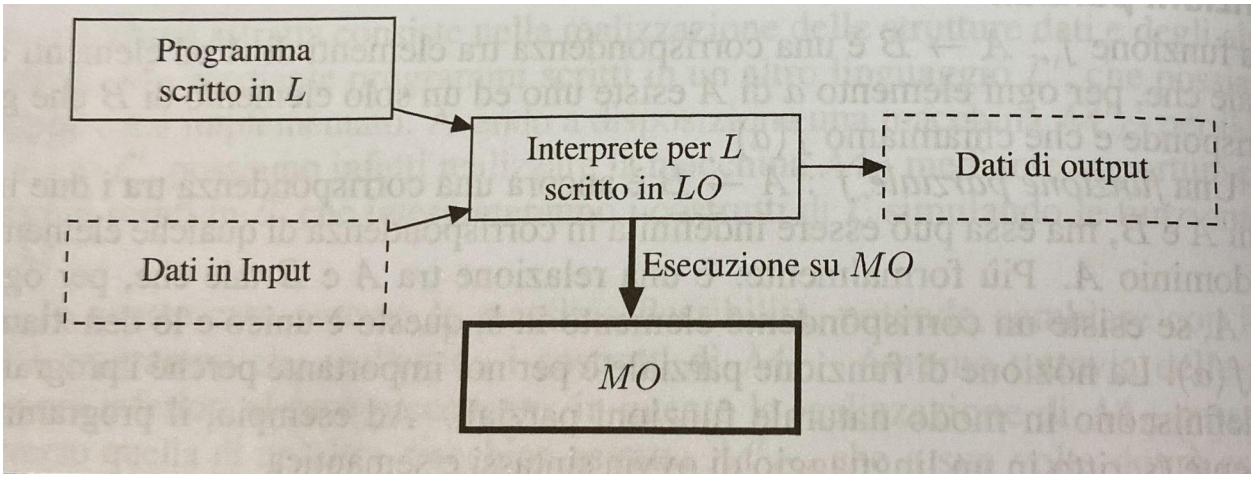
Implementazione interpretativa pura

Si realizza l'interprete di M_L mediante un insieme di istruzioni in Lo, si realizza cioè un programma che è un interprete I_L^{Lo}.

Per poter eseguire un programma P^L bisognerà eseguire sulla macchina Mo_{Lo} il programma I_L^{Lo}, con il programma e i dati come input.

Il fatto che un programma possa essere considerato come un dato in input per un altro programma non deve meravigliare, visto che abbiamo detto che è anche lui una sequenza di bit

Nell'implementazione interpretativa quindi non c'è una traduzione esplicita dei programmi, ma solo un procedimento di "decodifica" perchè il codice corrispondente ad un'istruzione di L viene eseguito, e non prodotto in uscita dall'interprete



Implementazione compilativa pura

L'implementazione di L avviene traducendo esplicitamente i programmi scritti in L in programmi scritti in Lo, e questa traduzione viene eseguita dal compilatore $C_{L,Lo}$.

In questa implementazione, L è detto linguaggio sorgente, mentre Lo è detto linguaggio oggetto.

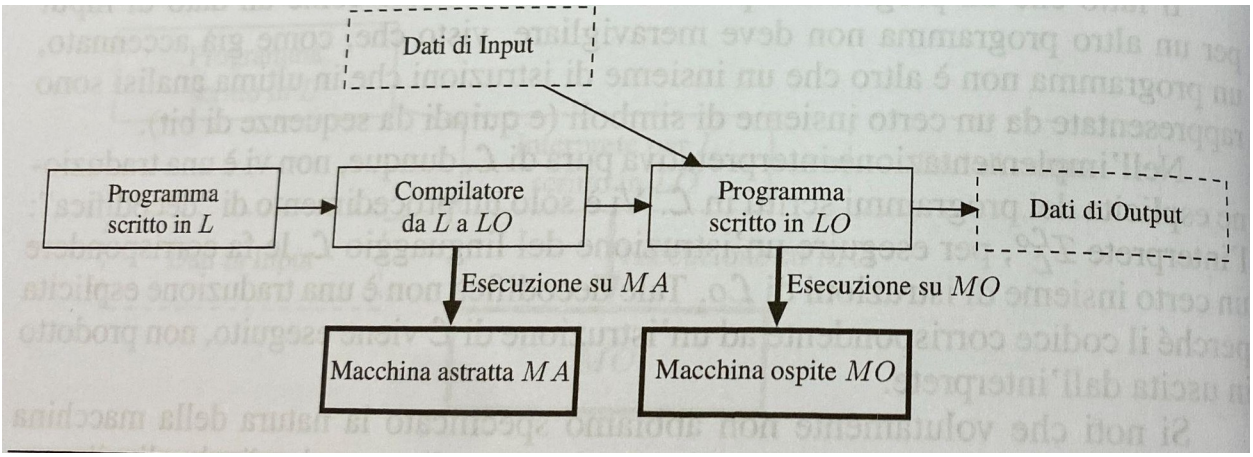
Quindi per poter eseguire un programma P^L bisognerà prima eseguire $C_{L,Lo}$ con P^L come input, a questo punto l'output che sarà Pc^{Lo} sarà eseguito normalmente sulla macchina Mo_{Lo} con i dati in input.

In questo caso la fase di traduzione è separata dalla fase di esecuzione.

Il compilatore potrebbe essere eseguito anche su una macchina che usi un linguaggio diverso da Lo, ma che comunque produca un programma scritto in Lo

I compilatori solitamente non compilano il programma totalmente, certe parti del programma potrebbero venir compilate separatamente oppure essere già state compilate (librerie).

Il linker è il programma che mette assieme tutte le parti del programma compilate



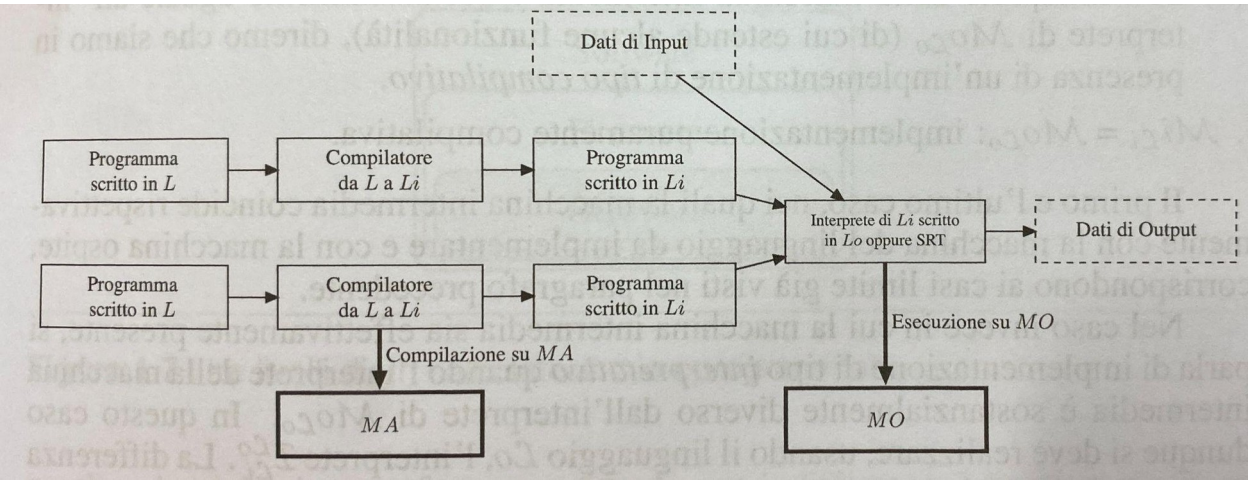
Confronto fra le due

La implementazione interpretativa ha lo svantaggio della scarsa efficienza, visto che la traduzione viene fatta a tempo di esecuzione e, ad esempio, ogni occorrenza di uno stesso comando viene decodificata singolarmente ogni volta.
Come vantaggio però c'è la flessibilità, in quanto la traduzione durante l'esecuzione è molto utile per operazioni di debugging e la implementazione di un interprete è più semplice e breve di quella di un compilatore, e risparmia memoria, in quanto il programma è memorizzato solo nella sua versione sorgente

Invece per l'implementazione compilativa, l'esecuzione risulta più veloce, e una volta fatta una decodifica di una istruzione, può venire riutilizzata per le successive occorrenze.
Uno svantaggio è quello della perdita di informazioni del programma sorgente

Implementazione: il caso reale e la macchina intermedia

Nel caso reale si preferisce implementare una macchina astratta mediante una combinazione dell'implementazione interpretativa e quella compilativa.



In questa situazione ci sarà appunto un linguaggio intermedio Li e una macchina intermedia Mi_{Li} e di conseguenza avremo un compilatore $C_{L,Li}$ e un interprete I_{Li}^{Lo}

Gerarchie di macchine astratte

Si possono creare infiniti livelli di macchine astratte, infatti se scriviamo un programma P in un linguaggio L tale che il programma P sfrutti un nostro “linguaggio” Lp, abbiamo praticamente creato una nuova macchina astratta.
Quindi possiamo ipotizzare una gerarchia di macchine $M_{L_0}, M_{L_1}, \ldots, M_{L_n}$ tali che ogni macchina M_{L_i} è implementata sfruttando le funzionalità della macchina sottostante $M_{L_{i-1}}$ e fornisce il proprio linguaggio L_i alla macchina sovrastante $M_{L_{i+1}}$
Questo viene fatto anche con lo scopo di mascherare i livelli inferiori, per negare l’accesso a risorse delle macchine sottostanti

I nomi e l’ambiente

Nomi e oggetti denotabili

Un name è una sequenza di caratteri usata per rappresentare qualcosa (nome di una costante, di una variabile, di una funzione)
Un nome dunque non è altro che una sequenza di caratteri usati per rappresentare, o denotare, un altro oggetto
Un nome e l’oggetto che rappresenta non sono la stessa cosa, un nome è solo una stringa, la rappresentazione può essere su un oggetto complesso. Un singolo oggetto può avere più nomi (aliasing), un singolo nome può rappresentare differenti oggetti in momenti diversi.
I nomi realizzano una prima **astrazione sui dati**, in quanto il nome nasconde i dettagli di basso livello.
L’**ambiente** è quella parte dell’implementazione che è responsabile delle associazioni tra i nomi e gli oggetti

I nomi poi sono fondamentali per realizzare una forma di **astrazione sul controllo**, in quanto il nome di una funzione nasconde tutto quello che avviene al suo interno in un certo senso

Oggetti denotabili

Gli **oggetti denotabili** sono tutte le cose a cui si può assegnare un nome.
Esistono due grandi tipologie di nomi:

- Definiti dall’utente
  Variabili, parametri formali, procedure, tipi definiti dall’utente, moduli, costanti definite dall’utente, eccezioni
- Definiti dal linguaggio
  Tipi primitivi, operazioni primitive (+ ad esempio), costanti predefinite

Binding (o associazione) è il termine tecnico usato per il legame tra nome e oggetto:
Esistono diverse fasi di binding, in base a in quale punto viene effettuato:

- Progettazione del linguaggio: per esempio + per l'addizione e ∫ ¿ per rappresentare il tipo integer
- Scrittura del programma: il programmatore sceglie i nomi quando scrive il programma, anche se l'associazione verrà "creata" solamente quando verrà allocata la memoria
- Compilazione (compile-time): variabili globali e statiche
- Esecuzione (run-time): variabili locali e dinamiche

Esistono anche fase di linking e loading per variabili provenienti dall'esterno.

Ogni cosa che viene fatta prima dell'esecuzione si dice statica, altrimenti, durante l'esecuzione si dice dinamica.
La gestione della memoria statica viene fatta dal compilatore, invece la gestione della memoria dinamica viene fatta da opportune operazioni a runtime

Ambiente e blocchi

Visto che le associazioni tra nomi e oggetti denotabili non sono fissate e immutabili, ma possono variare durante l'esecuzione, bisogna introdurre il concetto di ambiente.
L'ambiente (environment) è la collezione di associazioni (di binding) esistenti a runtime/attive in un specifico punto del programma in un specifico momento dell'esecuzione.
In qualsiasi momento del programma si deve poter dire quale è e da cosa è composto l'ambiente.
Una dichiarazione introduce un'associazione in un ambiente.
Alcuni linguaggi permettono dichiarazioni implicite, che introducono un'associazione in un ambiente al momento del primo uso di tale nome

Uno stesso nome può rappresentare differenti oggetti in differenti punti del programma, e uno stesso oggetto può essere denotato da più nomi: l'**aliasing** è il dare più nomi, visibili nello stesso momento, allo stesso oggetto (puntatori, riferimenti)

Blocchi

Nei linguaggi di programmazione moderni, l'ambiente è strutturato, un blocco è una sezione del programma che ha un ben definito inizio e una ben definita fine che contiene dichiarazioni locali a quella regione.

Tipi di ambiente

Solitamente il cambio di blocco è sinonimo di cambio di ambiente.
I blocchi, nei linguaggi in cui è concesso aprirne uno dentro ad un altro, devono essere annidati, ossia la definizione di un blocco deve essere interamente inclusa in quello di un altro
Si chiamano regole di visibilità quei meccanismi che regolano come e quando una dichiarazione è visibile. Ad esempio una dichiarazione in un blocco interno ad un altro non sarà visibile da quello esterno, ma nel caso contrario, la dichiarazione esterna sarà visibile dall'interno, e così via.
L'ambiente associato ad un blocco è costituito da:

- Ambiente locale
  Solo le dichiarazioni presenti nel blocco
- Ambiente non locale
  Dichiarazioni presenti in blocchi esterni
- Ambiente globale
  Dichiarazioni globali o statiche

Operazioni sull'ambiente

Ogni volta che si entra in un blocco vengono create le associazioni locali e vengono disattivate le associazioni dei nomi che sono stati riutilizzati localmente. All'uscita del blocco viene fatto il contrario.
Più in generale si possono identificare le seguenti operazioni sui nomi e sull'ambiente:

- Naming
  Creazione di un associazione fra nome e oggetto denotato
- Referencing
  Riferimento di oggetto denotato mediante il suo nome
- Disactivation of an association
- Reactivation of an association
- Unnaming
  Destruction of an association

Se si fa un binding "più interno" non si distrugge quello esterno ma viene nascosto finchè non esiste più quello interno, quindi l'ambiente contiene sia associazioni attive che associazioni disattivate (temporaneamente)

Operazioni su un oggetto rappresentabile (denotabile):

- Creazione
  Allocazione della memoria
- Accesso
  Tramite il nome
- Modifica
  Sempre tramite il nome
- Distruzione
  Deallocazione della memoria

La creazione e la distruzione di un oggetto non è la stessa cosa della creazione e la distruzione di un link su quell'oggetto

Lifetime di un oggetto:

1. Creazione dell'oggetto
2. Creazione di un'associazione
3. Reference sull'oggetto, tramite l'associazione
4. Disattivazione dell'associazione
5. Riattivazione dell'associazione
6. Distruzione dell'associazione
7. Distruzione dell'oggetto

Notare che dal punto 2 al punto 6 abbiamo il ciclo di vita dell'associazione

La lifetime di un oggetto può essere più lunga della lifetime di una associazione a quell'oggetto (ad esempio quando si passa una variabile per riferimento), ma può essere anche più corta (dangling reference, ad esempio per la memoria dinamica)

Regole di scope

- Scoping statico (o lessicale)
  Dipende dalla struttura sintattica del programma, dipende dal blocco che sintatticamente include la variabile
- Scoping dinamico
  Dipende dalla sequenza di istruzioni che viene eseguita, dall'ordine, dipende dall'ultimo blocco che è stato "attivato" (e non ancora disattivato)

Scope statico

- Le informazioni si determinano dal testo del programma
- Facile da implementare, più leggibile e più efficiente
- La regola dello scoping statico è definita dai tre punti
  1. Le dichiarazioni locali di un blocco definiscono l'ambiente locale, non includono le dichiarazioni presenti in sottoblocchi annidati
  2. Se si usa un nome all'interno di un blocco, l'associazione valida è quella locale del blocco, se non esiste vale quella locale del blocco immediatamente esterno, se non esiste si prosegue avanti con i blocchi esterni, se non esiste in nessun blocco allora si cerca nell'ambiente predefinito dal linguaggio, e se non esiste neanche qua si ha un errore
  3. Un blocco può avere un nome, in tal caso questo nome fa parte dell'ambiente del blocco immediatamente esterno, quello che contiene questo blocco con il nome (essendo visibile dal blocco esterno, sarà visibile anche all'interno del blocco col nome, questo permette le procedure ricorsive)

Scope dinamico

- Le informazioni si hanno durante l'esecuzione
- Difficile da leggere, difficile da implementare e poco efficiente
- Implementato con una politica LIFO
- La regola dello scoping dinamico:
  - L'associazione valida per un nome X in un qualsiasi punto del programma è la più recente (in senso temporale) associazione creata per X che sia ancora attiva al momento dell'associazione

Notiamo che le differenze tra scoping statico e dinamico sono solo per la determinazione dell'ambiente che è non locale e non globale, perché per variabili locali e globali le regole sono le stesse
In C non è possibile annidare blocchi, questo rende più facile lo scoping

L'environment quindi è determinato da:
- Regole di scoping
- Regole specifiche
- Regole di passaggio di parametri
- Regole di binding

Come si fa a definire una funzione ricorsiva se non è stata dichiarata prima?
- Java permette di chiamare funzioni dichiarate successivamente nel codice
- In C prima si dichiara e poi si definisce

La gestione della memoria

Nel caso di una macchina astratta di basso livello la gestione della memoria è semplice e interamente statica, invece in un linguaggio ad alto livello la questione è più complicata, soprattutto se il linguaggio permette la **ricorsione**.
Infatti in un programma senza ricorsione, possiamo stabilire il numero delle procedure a priori, questo con la ricorsione non è più vero perchè dipende da molti fattori (da informazioni disponibili solo a run-time)
Esistono tre metodi per l'allocazione della memoria:
- Static
  Memoria allocata a compile-time dal compilatore
- Dynamic
  Memoria allocata a run-time
  - Stack
    Allocata in maniera LIFO
  - Heap
    La memoria può essere allocata e deallocata in qualsiasi momento

Gestione statica

Nella allocazione statica gli oggetti hanno un indirizzo assoluto (risiedono in una zona fissa di memoria) che vale per tutta l'esecuzione del programma (variabili globali, istruzioni del codice oggetto, costanti, le tabelle di garbage collection, gestione dei nomi etc)
Nel caso in cui il linguaggio non supporti la ricorsione, è possibile associare staticamente ad ogni procedura una zona di memoria (formata da variabili locali, parametri, indirizzo di ritorno, valori temporanei). Questo funziona perché due chiamate diverse della stessa procedura, anche se condividono la stessa memoria, non creano problemi (**in assenza di ricorsione**).
L'allocazione statica non permette la ricorsione, perché se lo permettesse i valori delle nuove chiamate sovrascriverebbero quelli vecchi.

Gestione dinamica tramite stack

La allocazione dinamica si basa su una struttura a blocchi con una conseguente politica LIFO
Ogni volta che si entra in un blocco viene eseguita una $push()$ nella pila, e ogni volta che si esce viene eseguito una $pop()$
Lo spazio di memoria, allocato sulla pila, dedicata a un blocco o una procedura si chiama **record di attivazione** (RdA) o frame. Da notare che esistono RdA distinte e diverse per ogni chiamata di una stessa procedura.
Da notare che in certi casi conviene usare questo tipo di gestione di memoria anche se il programma non supporta la ricorsione, perché permette di risparmiare memoria

RdA per i blocchi in-line

Le RdA per un blocco in-line comprende le seguenti informazioni:

- Risultati intermedi
  Ad esempio per alcuni valori intermedi ai quali il programmatore non assegna un nome esplicito
- Variabili locali
  La dimensione dipende dal numero e dal tipo di variabili
- Puntatore di catena dinamica
  Questo campo serve per memorizzare il puntatore al precedente RdA sulla pila, viene anche chiamato link dinamico o link di controllo
  L'insieme dei collegamenti realizzati da questi puntatori è detto **catena dinamica**

Esiste uno stack pointer che punta all'ultimo blocco, e riesce a risalire ai blocchi precedenti per variabili dichiarate in blocchi superiori
Nella maggior parte dei linguaggi i blocchi in-line possono essere dichiarati senza uno stack, questa cosa causa uno spreco di memoria ma non perde tempo e risorse per la gestione dello stack

RdA per le procedure

Il caso delle procedure e delle funzioni è analogo al precedente, ma con qualche complicazione in più dovuta al fatto che servono più informazioni.
L'unica differenza tra procedure e funzioni è che le funzioni hanno un valore di ritorno, quindi ci sarà bisogno di tener conto della locazione di memoria nella quale memorizzare il valore restituito
Le RdA per una procedura o funzione comprende le seguenti informazioni:

- Risultati intermedi, variabili locali, puntatore di catena dinamica
  Uguali a quelli precedenti
- Puntatore di catena statica
  Serve per gestire le informazioni a realizzare le regole di scope statico
- Indirizzo di ritorno
  Contiene l'indirizzo della prima istruzione da eseguire dopo che la procedura/funzione ha terminato l'esecuzione

- Indirizzo del risultato
  Solo per le funzioni, si tratta dell'indirizzo (presente nella RdA del chiamante) di una variabile in cui bisogna inserire il valore di ritorno della funzione
- Parametri
  Il valore dei parametri attuali usati nella chiamata

Il puntatore di catena dinamica punta a una zona fissa della RdA superiore, e tramite un offset si possono raggiungere gli indirizzi dei vari campi. Gli indirizzi delle variabili locali hanno una posizione "fissa" all'interno dell'RdA perchè le dichiarazioni all'interno di un blocco hanno un ordine fissato, e quindi è possibile sapere con esattezza in che posizione dell' RdA si trova un campo

Gestione della pila

Per potersi riferire alla pila c'è bisogno di un puntatore esterno che punti all'ultima RdA inserita nella pila, questo puntatore si chiama puntatore al record di attivazione. (In certe implementazioni esiste un altro puntatore che indica la prima posizione libera nella pila)

La gestione della pila viene fatta sia dal chiamante che dal chiamato, il compilatore infatti inserisce dei pezzi di codice nel chiamante e nel chiamato per gestire correttamente tutte le operazioni che vengono fatte sulla pila e all'interno delle RdA. Queste attività comprendono:

- Modifica del valore del PC/IP
  Bisogna memorizzare l'indirizzo della istruzione da eseguire dopo la fine della funzione, il quale viene salvato in indirizzo di ritorno
- Allocazione dello spazio sulla pila
  Predisporre lo spazio per la nuova RdA e cambiare il puntatore alla prima posizione libera della pila
- Modifica del puntatore al RdA
  Il puntatore dovrà indicare il nuovo RdA appena inserito
- Passaggio dei parametri
  Compito del chiamante, visto che chiamate diverse hanno parametri diversi
- Salvataggio dei registri
  I valori per la gestione del controllo, ad esempio il vecchio puntatore RdA deve essere salvato come puntatore di catena dinamica
- Esecuzione del codice per l'inizializzazione

Invece al momento della fine dell'esecuzione della funzione devono essere svolte le seguenti attività:

- Ripristino del valore del contatore programma
  Per restituire il "controllo" al chiamante
- Restituzione dei valori
  Il valore di ritorno viene salvato all'indirizzo puntato da indirizzo del risultato
- Ripristino dei registri
  Viene ripristinato il vecchio valore del puntatore al RdA
- Esecuzione del codice per la finalizzazione
- Deallocazione dello spazio sulla pila
  Con conseguente modifica del puntatore alla prima posizione libera della pila

Gestione dinamica mediante heap

Esistono dei metodi di allocazione di memoria che non seguono un'ordine implementabile con una pila, questa allocazioni che possono avvenire in istanti di tempo arbitrari vengono fatte nella heap.
I metodi di gestione dello heap si dividono in due categorie principali:

- Blocchi di memoria a dimensione fissa
- Blocchi di memoria a dimensione variabile

Blocchi di dimensione fissa

In questo caso lo heap è diviso in blocchi di dimensione fissa, collegati in una struttura a lista, detta lista libera.
Quando viene effettuata un'allocazione, il primo elemento della lista libera viene rimosso da essa e il puntatore a tale elemento viene restituito all'operazione che aveva chiesto la memoria, in più la lista libera viene aggiornata in modo tale da puntare all'elemento successivo.
Nel momento in cui si effettua la deallocazione, il blocco collegato viene nuovamente messo in testa alla lista libera

Blocchi di dimensione variabile

Nel caso in cui il linguaggio permetta allocazioni di memoria ad oggetti di dimensione variabile (ad esempio array dinamici), si usa una gestione dello heap con blocchi di dimensione variabile.
Essendo più complicato questo tipo di gestione usa tecniche diverse per ottimizzare le operazioni.
In particolare si cerca di evitare la frammentazione interna (quando si alloca un blocco di dimensione strettamente maggiore a quella richiesta dal programma) perché la porzione di memoria non utilizzata verrà sprecata fino alla deallocazione.
Ma è ben peggiore il caso della frammentazione esterna, dove la lista libera è composta da blocchi relativamente piccoli, quindi anche se la somma della memoria libera è più che sufficiente, è inutilizzabile.
Quindi per evitare questi problemi le tecniche di allocazione di memoria tendono a "ricompattare" la memoria libera, unendo blocchi liberi contigui

Unica lista libera

Questa tecnica consiste nel considerare un'unica lista libera, che contiene un unico grande blocco di memoria. Appena viene richiesta un'allocazione di n, le prime n parole vengono allocate, e il puntatore all'inizio dello heap si sposta di n.
Al momento della deallocazione i blocchi vengono inseriti in una lista libera.
Nel momento in cui si finisce lo spazio di memoria dello heap, si passa allo spazio di memoria deallocato gestito dalla lista libera in due modi:

i. Utilizzo diretto della lista libera
   In questo caso viene mantenuta una lista libera di dimensione variabile, e quando si richiede l'allocazione per un blocco di n parole, viene cercato un blocco di dimensione maggiora o uguale a n o in maniera first fit (il primo trovato) o in maniera best fit (il più

piccolo compatibile) e la differenza di parole tra il blocco trovato e quello da allocare, se superiore ad un limite prefissato (sotto tale soglia è ammissibile la frammentazione interna) va a costruire un nuovo blocco e viene inserito nella lista libera.
Infine viene fatta una compattazione parziale, nel senso che vengono "ricompattati" due blocchi liberi solo se adiacenti (per ridurre la frammentazione esterna)

ii. Compattazione della memoria libera
Secondo questa tecnica quando si arriva alla fine dello heap, tutti i blocchi non ancora deallocati vengono spostati ad un'estremità della heap, lasciando così tutta la memoria libera disponibile in un unico blocco contiguo, e questa operazione viene effettuata ogni volta che si arriva alla fine dello heap

Liste libere multiple

Per ridurre il costo di allocazione di un blocco alcuni metodi usano più liste di dimensione diversa, in questo modo quando viene richiesta l'allocazione di dimensione n viene selezionata la lista con blocchi di dimensione maggiore o uguale a n.
Le divisioni dei blocchi possono essere statiche o dinamiche.
Due metodi per la divisione dinamica sono:

- buddy system
  m liste e ogni lista ha blocchi di grandezza 2^m
  Nel caso in cui tutti i blocchi della lista k siano occupati, si divide in due un blocco della lista $k+1$ e la metà che resta libera, che avrà dimensione 2^k viene inserito nella lista k.
  Nel momento della deallocazione di uno dei due blocchi, viene controllato se il "compagno" (buddy) è libero, e in questo caso vengono uniti e rimessi nella lista $k+1$
- heap di Fibonacci
  Logica identica al precedente solo che essendo che fibonacci sale più lentamente della potenza di 2, ci sarà una minore frammentazione interna

Implementazione delle regole di scope

La possibilità di denotare oggetti, anche complessi, medianti nomi con opportune regole di visibilità è un aspetto importantissimo che differenzia un linguaggio di alto livello da uno di basso livello. La realizzazione dell'ambiente e delle regole di scope viste nel capitolo sui Nomi e l'Ambiente richiedono opportune strutture dati aggiuntive.
Il tipo di scope definisce l'ordine con cui esaminare i RdA.

Scope statico

Con la regola di scope statico, l'ordine con cui esaminare i RdA non è quello definito dalla loro posizione nella pila, ma il primo RdA in cui cercare sarà definito dalla struttura testuale del programma.

La catena statica

I puntatori di catena statica servono proprio a questo, i puntatori di catena statica sono **fissi** per ogni istanza della procedura e puntano sempre alla RdA dove la procedura è stata dichiarata.

Quindi il puntatore di catena dinamico cambia in base a dove è chiamata la funzione, il puntatore di catena statico è fisso.
Il calcolo del puntatore di catena statica è semplice e può essere diviso in due casi:

- Il chiamato si trova all'esterno del chiamante
- Il chiamato si trova all'interno del chiamante

Il compilatore usa la tabella dei simboli, una sorta di dizionario dove il compilatore memorizza tutti i nomi(tipo, visibilità) degli oggetti, e vengono identificati e numerati i vari scope in base al livello di annidamento e si segna il numero "di blocchi" da superare per ogni variabile

Il display

L'inefficienza della catena statica è che per ogni variabile non locale dobbiamo effettuare k accessi in memoria per scorrere la catena statica.
La tecnica del display invece permette di ridurre questo k a 2.
Questa tecnica usa un vettore, detto display, che contiene n elementi, dove n è il numero di livelli di annidamento dei blocchi presenti nel programma.
Quindi quando ci si riferisce ad un oggetto non locale, dichiarato in un blocco di livello k, basta accedere alla posizione k del vettore e seguire il puntatore presente in tale posizione.
La gestione del display è semplice anche se leggermente più costosa in quanto quando si entra in un ambiente, oltre ad aggiornare il puntatore presente nel vettore, si deve anche salvare il vecchio valore.

Scope dinamico

Concettualmente, l'implementazione di uno scoping dinamico è molto più semplice in quanto basterebbe risalire la pila nell'ordine in cui è già creata.

Lista di associazioni

Alternativamente alla memorizzazione diretta nelle RdA, le associazioni nomi-oggetto possono essere memorizzate separatamente in una lista di associazioni detta A-list, gestita come una pila. Ogni volta che si entra in un ambiente le associazioni locali vengono inserite nella A-list, e vengono tolte ogni volta che si esce da un ambiente.
Sia l'uso diretto delle RdA sia la A-list però presentano due inconvenienti.

- Non sappiamo mai quale sia il blocco dove possiamo risolvere un riferimento non locale
- Nel caso di una variabile dichiarata nei primi blocchi bisogna scorrere tutta la lista, ed è inefficiente

CRT

Per limitare questi due inconvenienti, al prezzo di una maggiore inefficienza per le operazioni di entrata e uscita dai blocchi, possiamo usare una implementazione basata sulla Tabella Centrale dell'Ambiente o CRT (Central Referencing environment Table).
In tale tabella sono presenti tutti i nomi usati nel programma e per ogni nome è presente un flag, che indica se l'associazione per quel nome è attiva o meno.
Uno dei metodi per implementarla è che ogni nome ha una propria lista, in cui il primo elemento è l'ultimo attivo, ossia quello valido.

Astrarre sul controllo

Astrarre significa in un certo senso nascondere qualcosa, infatti per quanto riguarda le macchine astratte, le quali abbiamo messo in contrapposizione con quelle fisiche, nascondono qualcosa che si trova al di sotto di loro.
L'astrazione consiste nel "concentrarsi" sugli aspetti più rilevanti, nascondendo gli altri; quello che è rilevante dipende dall'obiettivo del progetto (dall'obiettivo del linguaggio).

Sottoprogrammi

Costruire vari sottoprogrammi che, messi assieme, formano un programma più complesso è molto comodo ed efficace, però serve un supporto linguistico che permetta e faciliti questa suddivisione e la successiva ricomposizione.
Questi sottoprogrammi (o funzioni, o procedure) sono delle porzioni di codice, identificate da un nome, dotate di ambiente locale proprio e capace di scambiare informazioni attraverso i parametri (input), il valore di ritorno (output) e l'ambiente non locale
Una funzione viene dichiarata e poi chiamata.

I **parametri** si distinguono in:

- parametri formali
  Quelli che compaiono nella definizione della funzione, non hanno senso al di fuori di essa
- parametri attuali
  Quelli che compaiono nella chiamata della funzione, possono essere anche espressioni e non solo nomi

Alcune funzioni scambiano informazioni col resto del programma anche restituendo un **valore come risultato** della funzione stessa (ad esempio col costrutto *return*).
In genere si fa una distinzione tra funzione, i sottoprogrammi che restituiscono un valore, e procedura, i sotto programmi che interagiscono col chiamante solo tramite parametri e ambiente non locale

Per quanto riguarda **l'ambiente non locale**, si tratta del meccanismo meno soffisticato in quanto una modifica di una variabile non locale si ripercuote in tutte le porzioni del programma dove quella variabile è visibile.

Astrazione funzionale

I sottoprogrammi sono meccanismi che permettono di ottenere una astrazione funzionale, in quanto i "clienti" di tale componente non hanno interesse sul **come** tali servizi vengono implementati, ma solo sul come richiederli e a cosa servono.

Infatti il cliente ha bisogno di sapere solo l'intestazione (nome, parametri, eventuale risultato) per poter usare una componente.
Questa pratica permette di poter modificare e aggiornare il codice di una sottofunzione, senza che lato cliente cambi qualcosa

Passaggio dei parametri

Le modalità con cui i parametri attuali vengono accoppiati ai parametri formali sono dette **modalità di passaggio dei parametri**.
Ogni modalità è costituita sia dal tipo di comunicazione che permette di ottenere, sia dall'implementazione usata per ottenerla, la modalità viene fissata al momento della definizione della funzione
La classificazione del tipo di comunicazione permessa da un parametro è semplice, potendo individuare tre semplici classi di parametri:

- Parametri di ingresso
- Parametri di uscita
- Parametri sia di ingresso che di uscita

Le caratteristiche delle modalità di passaggio sono le seguenti:

- quale tipo di comunicazione permette di stabilire
- quale forma di parametro attuale sia permessa
- la semantica della modalità
- l'implementazione canonica
- il suo costo

Le due modalità più importanti sono per valore e per riferimento, tutte le altre tecniche sono delle varianti del passaggio per valore, invece il passaggio per nome è più complesso (e in disuso).
Le modalità di passaggio dei parametri sono le seguenti:

- Passaggio per valore (default nella maggior parte dei linguaggi)
  Corrisponde ad un parametro di ingresso.
  Al momento della chiamata della funzione viene valutato il $r\text{-}value$ del parametro attuale e viene assegnato al parametro formale.
  Al termine della procedura il parametro formale viene eliminato, non c'è alcun legame tra parametro formale e parametro attuale, non c'è nessun modo per per trasferire informazioni dal chiamato al chiamante tramite parametro attuale.
  È una modalità molto costosa in quanto nel caso in cui il parametro attuale corrisponda a una struttura dati di grandi dimensioni.
- Passaggio per riferimento
  Corrisponde ad un parametro sia di ingresso che di uscita
  Il parametro attuale **deve** essere una espressione dotata di $l\text{-}value$, e c'è un aliasing del parametro attuale, infatti c'è una associazione tra parametro formale e $l\text{-}value$ del parametro attuale.
  Ogni modifica del formale è una modifica dell'attuale, consiste nel dare un altro nome al parametro attuale, che però vive solo nella funzione.

È una modalità di costo molto contenuto, in quanto bisogna solo memorizzare un indirizzo

- Passaggio per costante
  Implementa la semantica del passaggio del valore (quindi solo in input) però implementandola attraverso il passaggio per riferimento (read only).
  In realtà l'implementazione può essere a discrezione del compilatore (per dati di piccole dimensioni viene fatta per valore, per grandi dimensioni per riferimento)
- Passaggio per risultato
  È l'esatto contrario del passaggio per valore, consiste in una modalità di sola uscita.
  Il parametro attuale deve essere una espressione dotata di $l\text{-}value$ e al momento della terminazione della funzione, il valore corrente del parametro formale viene assegnato alla locazione ottenuta mediante lo $l\text{-}value$ del parametro attuale.
  Durante l'esecuzione del corpo non c'è alcun legame tra il parametro formale e l'attuale e non c'è nessun modo di trasferire informazioni dal chiamante al chiamato tramite esso.
- Passaggio per valore-risultato
  La combinazione del passaggio per valore e il passaggio per risultato.
  La differenza col passaggio per riferimento è che lo scambio di informazioni avviene solo in due momenti, alla chiamata e al termine della funzione, quindi nel corpo non c'è modo di scambiare dati da una parte all'altra (mentre per riferimento essendo aliasing c'è uno scambio continuo).
- Passaggio per nome
  Il passaggio per nome consiste nel prendere il codice della funzione, e letteralmente sostituire ogni occorrenza del parametro formale con il parametro attuale.
  Questo però può causare errori nel caso in cui un nome venga usato per altre variabili all'interno della funzione, e quindi si è risolto facendo la valutazione del parametro attuale a ogni occorrenza del parametro formale, distinguendolo da quello locale.
  Un altro problema è che essendo valutato in ogni occorrenza, se il parametro attuale è una espressione del tipo $i++$, allora a ogni occorrenza verrà incrementato i, causando effetti collaterali.
  Nel passaggio per nome c'è il bisogno di passare, oltre al parametro attuale, anche l'ambiente, cosicché ad ogni occorrenza del parametro formale, venga valutato il parametro attuale in base all'ambiente (con static scoping), in realtà vengono passati un puntatore all'espressione del parametro attuale e un puntatore alla catena statica del chiamante.
  Si tratta di una modalità molto complessa e costosa, dovendo valutare ripetutamente un parametro in un ambiente diverso da quello corrente.

Funzioni di ordine superiore

Una funzione è di **ordine superiore** quando ha o come parametro o come valore di ritorno un'altra funzione. Linguaggi che permettano di passare funzioni come parametri sono abbastanza diffusi, sono meno diffusi invece quelli in cui è permesso il valore di ritorno come funzione.

Avere una funzione come valore di ritorno è alla base della programmazione funzionale, e ci sono molti problemi linguistici e implementativi di questa pratica.

Funzioni come parametro

In questo caso si possono dichiarare funzioni che hanno come parametro un'altra funzione, ad esempio la funzione *void* $g\ \left(\int\ h\left(\int\ n\right)\right)\{\}$ è una funzione che prende come unico parametro una funzione che ritorna un intero e che come parametro ha un altro intero.
Notiamo come il nome *n* sia completamente inutile, in quanto non ci sono modi con cui il programmatore possa utilizzarlo nel corpo di g.
Per usare quella funzione bisognerà chiamarla così $g(f)$ con f che deve essere una funzione definita così $\int\ f(\int\ y)\{\}$.

```
{int x = 1;                                    1
int f(int y){
     return x+y;                               3
}
void g (int h(int b)){                         5
     int x = 2;
     return h(3) + x;                          7
}
...                                            9
 {int x = 4;
  int z = g(f);                                11
 }
}                                              13
```

Deep binding: viene utilizzato l'ambiente attivo al momento della creazione del legame tra f e h (quindi quando viene chiamata g)
Shallow binding: viene utilizzato l'ambiente attivo al momento della chiamata di f tramite h
In questo esempio il nome x può essere valutato in più ambienti:

- Scope statico
  In questo caso viene usato solo il deep binding, essendo che il shallow binding sarebbe contraddittorio a livello metodologico
  - Deep binding
    Viene valutato al momento della chiamata di g (a riga 11) tramite scoping statico e il risultato è $z=6$
    La x nel corpo di f è quella del blocco più esterno (riga 1)
- Scope dinamico
  - Deep binding
    Viene valutato al momento della chiamata di g (a riga 11) tramite scoping dinamico e il risultato è $z=9$
    La x nel corpo di f è quella più recente al momento della creazione del legame (riga 10)
  - Shallow binding
    Viene valutato al momento della chiamata di f in g (a riga 7) tramite scoping

dinamico e il risultato è $z = 7$
La x nel corpo di f è quella più recente al momento della chiamata di h (riga 6)

L'implementazione del shallow binding non pone ulteriori problemi implementativi rispetto alle tecniche dello scope dinamico, in quanto anche in questo caso basta risalire lo stack.
Non è così invece per il deep binding che richiede strutture ausiliarie rispetto alla catena statica o dinamica.
Di solito la catena statica è fissa per ogni funzione e variabile, nel caso però di funzioni come parametri non sappiamo mai a quale funzione viene associato il parametro formale h, e quindi l'unico modo per ottenerlo è determinarlo al momento dell'associazione, ma deve essere associato anche l'ambiente dove il corpo di f deve essere valutato.
Quindi, come per il passaggio per nome, viene associata una **chiusura** (una coppia che consiste in puntatore al codice e puntatore al record di attivazione)

Quindi allarghiamo le regole che contribuiscono alla determinazione dell'ambiente:

- regole di visibilità
- le eccezioni alla regola di visibilità (ad esempio la possibilità di usare un nome prima della sua definizione)
- regole di scope
- regole relative alle modalità di passaggio dei parametri
- politica di binding

Funzioni come risultato

La possibilità di generare funzioni come risultato di altre funzioni permette la creazione dinamica di funzioni a tempo di esecuzione.
Ovviamente la funzione avrà un suo ambiente quindi quello che viene ritornato in realtà è una chiusura (funzione+ambiente).
Il problema però è che l'ambiente a cui fa riferimento un nome di una funzione ricevuta come valore di ritorno, potrebbe essere distrutto per la disciplina della pila.
Per risolvere questo problema si possono allocare tutti i record di attivazione nello heap, e lasciare al garbage collector il compito di deallocare quando non ci sono più riferimenti attivi.

Eccezioni

Un'eccezione è un evento particolare che si verifica durante l'esecuzione del programma e che non deve (non può) essere gestito dal normale flusso del controllo.
Si può trattare di un errore di semantica dinamica (divisione per zero, overflow, etc) oppure di una situazione nella quale il programmatore decide esplicitamente di terminare la computazione corrente.
Per gestire correttamente le eccezioni un linguaggio deve:

1. Specificare quali eccezioni sono gestibili e come possono essere definite
2. Specificare come un'eccezione può essere sollevata, cioè con quali meccanismi si provoca la terminazione eccezionale di una computazione

3. Specificare come un'eccezione possa essere gestita, cioè quali sono le azioni da compiere al verificarsi di un'eccezione e dove deve riprendere l'esecuzione del programma

La gestione di un'eccezione si sostanzia in due diversi costrutti:

- Un meccanismo per definire una capsula attorno ad una porzione di codice (il blocco protetto) con lo scopo di intercettare le eccezioni che si dovessero verificare all'interno della capsula stessa
- La definizione di un gestore dell'eccezione, in genere legato staticamente ad un blocco protetto, al quale trasferire il controllo quando la capsula intercetta un'eccezione

```
class EmptyExcp extends Throwable {int x=0;};

int average(int[] V) throws EmptyExcp(){
   if (length(V)==0) throw new EmptyExcp();
   else {int s=0; for (int i=0, i<length(V), i++) s=s+V[i];}
   return s/length(V);
};
...
try{...
    average(W);
    ...
}
catch (EmptyExcp e) {write('Array␣vuoto'); }
```

In questo esempio la prima linea è la definizione dell'eccezione.
La seconda linea è la definizione di una funzione nel corpo della quale potrebbero essere sollevate eccezioni, infatti con *throws* … avvisiamo che la funzione potrebbe riportare una terminazione anomala per via dell'eccezione.
Con la parola *throw* si solleva l'eccezione, il blocco protetto è all'interno di *try* e il gestore e il *catch*.
Quando l'eccezione viene intercettata da un *try*, il controllo viene passato a *catch*.
Dopo aver gestito l'eccezione l'esecuzione procede normalmente alla prima istruzione che segue il gestore stesso
Ci sono due aspetti importante sulla gestione delle eccezioni:

1. Un'eccezione non è un evento anonimo, infatti ha un nome che viene esplicitamente menzionato nel costrutto *throw*
2. Sebbene l'esempio non lo mostri, l'eccezione potrebbe inglobare un valore che permetta di "reagire" al momento dell'eccezione

Nel caso in cui l'eccezione non sia gestita all'interno della procedura in esecuzione, occorre risalire lungo la catena delle chiamate di procedura fino a quando non si incontra un gestore compatibile oppure si raggiunge il livello più alto, che fornirà un gestore default (terminazione con errore).
Da notare il fatto che le eccezioni si propagano lungo la catena dinamica, e quindi l'eccezione viene gestita il più vicino possibile a dove si è verificata.

Le eccezioni possono essere usate anche per rendere un programma più elegante ed efficiente, ad esempio una funzione che esegue il prodotto degli interi dei nodi di un albero binario. Questi nodi potrebbero avere degli 0 nei primi nodi, però l'albero potrebbe essere molto grande e potrebbe perdere molto tempo ad eseguire tutto lo scorrimento. Allora l'idea è che, dato che appena si incontra uno 0 il risultato sarà sicuramente 0, di mettere un eccezione nel caso in cui il valore di un nodo sia zero, e nel *catch* mettere *return* $0;$ così da ottimizzare il programma.

Implementare le eccezioni

Il modo più semplice e intuitivo con il quale una macchina astratta può implementare le eccezioni è quella di sfruttare la pila dei RdA, cioè ogni volta che si entra in un blocco protetto, viene inserito un puntatore al gestore corrente, e viene tolto ogni volta che si esce "normalmente".
L'inefficienza sta nel fatto che ad ogni entrata e uscita dal blocco protetto, bisogna manipolare la pila, anche nel caso in cui l'eccezione non si verifica.

Un altro metodo per implementarlo è creando a tempo di compilazione una tabella EH di indirizzi, nella quale per ogni blocco protetto si inseriscono ip (indirizzo blocco protetto) e ig (indirizzo gestore), così facendo per ogni eccezione sollevata a runtime, basta trovare la copia per la quale vale $ip \leq pc \leq ig$.
Essendo EH ordinata, si può effettuare una ricerca binaria (più efficiente).
Questa soluzione è più costosa per la ricerca del gestore (ma dipende dal numero di gestori), ma elimina il costo "costante" della modifica della pila ad ogni entrata e uscita da un blocco protetto, e siccome è ragionevole supporre che un'eccezione sia un evento più raro rispetto all'ingresso in un blocco protetto, si tratta di una soluzione mediamente più efficiente.

Strutturare i dati

Tipi di dato

Ogni linguaggio ha il suo **sistema di tipi** che lo differisce in maniera sostanziale dagli altri

Un tipo di dato è una collezione di valori omogenei ed effettivamente rappresentabili, dotata di un insieme di operazioni che manipolano tali valori
I tipi di dato hanno almeno tre scopi diversi:

- a livello di progetto, per l'organizzazione concettuale
- a livello di programma, per verificare la correttezza delle operazioni
- a livello di traduzione, per il supporto dell'implementazione (ottimizzazioni)

Tipi come supporto all'organizzazione concettuale

I tipi possono essere usati come strumento di documentazione, questo perché anche se due concetti possono essere entrambi rappresentati con lo stesso tipo, si possono definire nuovi tipi per separarli concettualmente e per assegnare loro tipi di operazione diversi.

In questo senso i tipi svolgono un ruolo analogo a quello dei commenti, con l'importante differenza che si tratta di commenti effettivamente controllabili.

Tipi per la correttezza

Ogni linguaggio ha le proprie regole di controllo dei tipi.
Tali vincoli sono presenti sia per evitare errori hardware a runtime e sia per evitare errori logici.
Molti linguaggio verificano che i vincoli siano tutti soddisfatti prima di procedere all'esecuzione (o alla generazione del codice) di un programma.
Questo è il ruolo del **type checker**, una componente del compilatore, il quale rileva e segnala ogni tentativo di uso improprio.
È evidente che un programma che rispetta tutte le regole di tipo può comunque essere logicamente scorretto, questo perchè i tipi assicurano una correttezza minimale, tuttavia fondamentale durante la fase di sviluppo.
Ci possono essere degli errori di tipo che non vengono rilevati dalla macchina astratta, per questo parleremo di linguaggi sicuri (type safe) e insicuri rispetto a questa possibilità.

Tipi e implementazione

La terza e ultima motivazione per l'uso dei tipi è che forniscono importanti informazioni per la macchina astratta, come ad esempio la dimensione della memoria da allocare.
Facilita anche l'accesso durante l'esecuzione, questo perché per fare l'accesso si applica un offset al puntatore dell'RdA, e sapendo a priori la dimensione dei tipi questa operazione diventa facile e statica.

Sistemi di tipi

Il sistema di tipi di un linguaggio è costituito da:

1. l'insieme dei tipi predefiniti dal linguaggio
2. i meccanismi che permettono di definire nuovi tipi
3. i meccanismi relativi al controllo dei tipi
    - regole di equivalenza (quando due tipi formalmente diversi corrispondono allo stesso tipo)
    - regole di compatibilità (quando il valore di un certo tipo può essere usato in un contesto che richiede un tipo diverso)
    - regole e tecniche di inferenza dei tipi (come il linguaggio attribuisce un tipo ad un'espressione complessa)
4. la specifica se i vincoli (e quali vincoli) siano da controllare staticamente o dinamicamente

Un sistema di tipi (e di conseguenza il suo linguaggio) è sicuro relativamente ai tipi quando nessun programma può violare le distinzioni tra tipi definite in quel linguaggio, in altre parole è sicuro quando nessun programma durante l'esecuzione può generare un errore non segnalato che derivi dalla violazione di un tipo.
Una classificazione dei tipi si può fare in base a come possono venire manipolati i suoi valori:

- denotabili (se possono essere associati ad un nome)

- esprimibili (se possono essere il risultato di un'espressione complessa, diversa da un semplice nome)
- memorizzabili (se possono essere memorizzati in una variabile)

Controlli statici e dinamici

Un linguaggio ha tipizzazione statica se i controlli dei vincoli di tipizzazione possono essere condotti a tempo di compilazione, sul testo del programma; altrimenti ha tipizzazione dinamica, cioè i controlli avvengono a runtime.
Il controllo dinamico è meno efficiente perché bisogna avere dei meccanismi che eseguono questi controlli a runtime, perdendo efficienza, e gli errori potrebbero venire fuori quando il programma è già operativo presso il suo utente finale.
Nel controllo statico, invece, il programma finale è garantito che sia esente da errori, visto che tutto è stato controllato a tempo di compilazione, di conseguenza l'esecuzione è più efficiente.
Però la progettazione di un linguaggio con controllo statico è molto più complessa rispetto a uno dinamico, poi la compilazione risulta più complessa e lenta (meglio compilazione lenta che esecuzione lenta però) e infine i controlli statici possono decretare come sbagliati programmi che in realtà non causerebbero errore a runtime (ad esempio un'assegnazione errata in una condizione che non si verificherà mai)
Infatti il controllore statico adotta una posizione prudenziale, cioè esclude più programmi di quelli che dovrebbe in modo da poter garantire la correttezza.
Come ogni cosa le soluzioni migliori sono una combinazione dei due.

Tipi scalari

Sono tipi scalari (o tipi semplici) tutti i tipi i cui valori non sono costituiti da aggregazioni di altri valori

Booleani

Il tipo dei valori logici (booleani) è costituito da:

- valori
  I due valori di verità, vero o falso
- operazioni
  Una selezione tra le principali operazioni logiche (and, or, not, uguaglianza, xor, etc)

Nei linguaggi dove è presente questo tipo (in C non è presente), i suoi valori sono memorizzabili, esprimibili e denotabili.
Per motivi di indirizzabilità, la rappresentazione in memoria consiste in un byte (e non un bit, anche se basterebbe quello)

Caratteri

Il tipo dei caratteri è costituito da:

- valori
  Un insieme di codici di caratteri (i più comuni sono ASCII e UNICODE)

- operazioni
  Fortemente dipendenti dal linguaggi, troviamo l'uguaglianza, i confronti e qualche modo per passare al carattere successivo/precedente

I suoi valori sono memorizzabili, esprimibili e denotabili.
La rappresentazione in memoria consiste in un byte (ASCII) o due byte (UNICODE)

Interi

Il tipo dei numeri interi è costituito da:

- valori
  Un sottoinsieme finito di numeri interi (di solito un intervallo della forma $\left[-2^t, 2^t-1\right]$)
- operazioni
  I confronti e un'opportuna selezione delle principali operazioni aritmetiche

I suoi valori sono memorizzabili, esprimibili e denotabili.
La rappresentazione in memoria consiste in un numero pari di byte (usualmente 2, 4 o 8) in complemento a 2

Reali

Il tipo dei numeri reali (o numeri a virgola mobile, float) è costituito da:

- valori
  Un opportuno sottoinsieme dei numeri razionali, la struttura (ampiezza, granularità, ect) dipende dalla rappresentazione adottata
- operazioni
  I confronti e un'opportuna selezione delle principali operazioni numeriche

I suoi valori sono memorizzabili, esprimibili e denotabili.
La rappresentazione in memoria consiste in 4, 8 o anche 10 byte in virgola mobile (secondo lo standard IEEE 754)

Virgola fissa

Il tipo dei numeri reali in virgola fissa è costituito da:

- valori
  Un opportuno sottoinsieme dei numeri razionali, la struttura (ampiezza, granularità, ect) dipende dalla rappresentazione adottata
- operazioni
  I confronti e un'opportuna selezione delle principali operazioni numeriche

I suoi valori sono memorizzabili, esprimibili e denotabili.
La rappresentazione in memoria consiste in 4 oppure 8 byte. I numeri sono rappresentati in complemento a 2 con un numero fissato di byte riservati per la parte decimale, consentono la rappresentazione di un ampio intervallo con pochi decimali di precisione

Complessi

Il tipo dei numeri complessi è costituito da:

- valori
  Un opportuno sottoinsieme dei numeri complessi, la struttura (ampiezza, granularità, ect) dipende dalla rappresentazione adottata
- operazioni
  I confronti e un'opportuna selezione delle principali operazioni numeriche

I suoi valori sono memorizzabili, esprimibili e denotabili.
La rappresentazione in memoria consiste in una coppia di valori in virgola mobile

Void

In alcuni linguaggi esiste un tipo primitivo che possiede un solo valore, talvolta indicato con $void$ (anche se sarebbe meglio chiamarlo $unit$, visto che non esistono funzioni con codominio vuoto)

- valori
  Uno solo, che possiamo indicare con $()$
- operazioni
  nessuna

Questo tipo serve per indicare il tipo di operazioni che modificano lo stato ma non restituiscono un valore

Enumerazioni

In aggiunta ai tipi predefiniti, troviamo nei linguaggi diversi modi per definire nuovi tipi.
Le enumerazioni e gli intervalli sono tipi scalari definiti dall'utente.
Una enumerazione consiste in un insieme finito di costanti, ciascuna caratterizzata dal proprio nome.
Le operazioni disponibili in un'enumerazione sono costituite dai confronti e da un meccanismo per passare al valore successivo/precedente (in questo senso si può considerare il tipo char un'enumerazione predefinita).
Le enumerazioni permettono la creazione di programmi molto leggibili, in quanto i nomi dei valori si auto-documentano.
Il valore di un enumerazione viene solitamente rappresentato con un intero su un byte.

Intervalli

I valori di tipo intervalli costituiscono un sottoinsieme contiguo dei valori di un altro tipo scalare (che sarà il tipo base dell'intervallo).
Il vantaggio di usare un tipo intervallo, come nelle enumerazioni, è che si ha una migliore documentazione e la possibilità di un controllo dei tipi più stringente.
Il valore di un intervallo viene solitamente rappresentato allo stesso modo in cui è rappresentato il tipo base dell'intervallo.

Tipi ordinali

I tipi dei booleani, dei caratteri, degli interi, le enumerazioni e gli intervalli sono esempi di tipi ordinali (o tipi discreti) e hanno le seguenti caratteristiche:

- Hanno una ben definita nozione di ordine totale

- Possiedono quindi una nozione di predecessore/successore
- Possono costituire il dominio su cui far variare gli indici di un vettore e sul quale definire delle iterazioni determinate

Tipi composti

I tipi non scalari sono detti composti, in quanto si ottengono combinando tra loro altri tipi mediante l'utilizzo di opportuni costruttori e sono:

- record (o strutture)
  Collezioni di valori eterogenei nel tipo
- array (o vettori)
  Collezioni di valori omogenei nel tipo
- insiemi
  Sottoinsiemi di un tipo base, in genere ordinale
- puntatori
  l-valori che permettono di accedere a dati di un altro tipo
- tipi ricorsivi
  Tipi definiti per ricorsione, usando costanti e costruttori (liste, alberi etc)

Record

Un record è una collezione costituita da un numero finito (e in genere ordinato) di elementi detti campi (fields) distinti dal loro nome. Ogni campo può essere di un tipo diverso dagli altri.
La sola operazione permessa su un valore di tipo record è in genere la selezione di una sua componente, di un suo campo.
In molti linguaggi è permessa la creazione di record annidati (cioè che un campo di un record può essere a sua volta un record).
L'uguaglianza e l'assegnazione tra record non è sempre consentita direttamente.
L'ordine dei campi è in genere significativo ed è rispettato nella rappresentazione in memoria.
Capita che ci siano dei "buchi" di memoria all'interno della rappresentazione in memoria di un record perché sono presenti campi di tipo diverso. Ed è per questo che alcuni linguaggi non assicurano che l'ordine dei campi venga mantenuto in memoria, per poter riordinare al meglio e lasciare meno buchi possibili, questa implementazione però, costa in termini di accesso.

Record varianti e unioni

Una forma particolare di record è quella in cui alcuni campi sono tra loro mutuamente esclusivi, in tal caso parliamo di record varianti.
In questa situazione ci sarà un tag (o discriminante) del record variante, e in base al suo valore (true o false) il record avrà dei campi diversi.
In un record variante solo una delle due varianti è significativa (non possono essere attive contemporaneamente), ed è per questo che condividono la stessa area di memoria.

```
type Stud = record
   nome : array [1..6] of char;
   matricola : integer;
   case fuoricorso : boolean of
      true: (ultimoanno : 2000..maxint);
      false:(inpari : boolean;
             anno : (primo,secondo,terzo)
             )
   end;
```

In certi linguaggi, come C, non esiste il concetto primitivo di record variante, ma quello di tipo unione.
Un'unione è analoga a un record, solo che solo uno dei campi di un'unione può essere attivo in un qualsiasi momento e i campi condividono la rappresentazione in memoria.

```
struct Stud{
   char nome[6];
   int matricola;
   int fuoricorso;
   union{
      int ultimoanno;
      struct{
         int inpari;
         int anno;
      } stud_in_corso;
   } campivarianti;
};
```

La differenza sostanziale tra record variante e la loro simulazione tramite le unioni è che ad esempio il campo fuoricorso nel caso delle unioni è un semplice campo, che non può influire con l'unione, e quindi potremmo avere degli errori logici (ad esempio uno studente fuoricorso che però ha dei dati in "inpari" e "anno").

Array

Un array (o vettore) è una collezione finita di elementi dello stesso tipi, indicizzata su un intervallo di tipo ordinale. Ogni elemento si comporta come una variabile del proprio tipo.
L'intervallo ordinale degli indici è il tipo indice.
Il tipo degli elementi è il tipo dell'array.
Tutti i linguaggi permettono la definizione di array multidimensionali, in altri linguaggi un array multidimensionale si ottiene dichiarando un array di array.
Alcuni linguaggi le due modalità sono equivalenti, altri ne accettano una sola delle due ed altri ancora le accettano tutte e due (con ad esempio dei vantaggi nell'array di array come lo slicing)

L'operazione più semplice permessa su un array è la selezione di un elemento, che si ottiene mediante un valore dell'indice.
Alcuni linguaggi permettono operazioni su array nella loro globalità (assegnamento, uguaglianza, confronti e operazioni aritmetiche).
Una slice (fetta) di un array è una sua porzione, costituita da elementi contigui.

La specifica del tipo indice di un array è parte integrante della sua definizione, infatti il controllo dei tipi del linguaggio dovrebbe verificare che ogni accesso ad un elemento sia davvero entro i limiti dell'array (che però può avvenire solo a tempo di esecuzione). Un linguaggio sicuro pertanto dovrà generare opportuni controlli in corrispondenza di ogni accesso.
Questo è un problema molto serio e infatti esistono degli attacchi hacker di questo tipo chiamato "buffer overflow" e consiste nel far leggere a un buffer un messaggio più lungo dell'area allocata per quel buffer, così da poter scrivere in zone di memoria dedicate ad altre cose (come valore o indirizzo di ritorno).

Un array di solito è memorizzato in una porzione contigua di memoria, per gli array multidimensionali si può avere una memorizzazione in ordine di riga (più comune) e una memorizzazione in ordine di colonna.
In base al tipo di memorizzazione conviene utilizzare un ciclo che segua quella sequenza, per ottimizzare il ciclo ed è importante nei sistemi con memoria cache.
Il calcolo dell'indirizzo di un elemento di un array non è difficile anche se richiede un po' di aritmetica. Facendo partire gli array da 0, l'aritmetica degli array diventa molto più facile e veloce.

La forma (o shape) di un array è costituita dal numero delle sue dimensioni e dall'intervallo su cui ciascuna di esse può variare.
In base ai tre modi su cui decidere quando fissare la forma di un array ci sono diverse modalità di allocazione:

- forma statica
  Tutto è deciso a tempo di compilazione e l'array può essere memorizzato nel RdA del blocco in cui compare la sua definizione.
  In questo caso la dimensione totale necessaria per un array è una costante nota a tempo di compilazione, pertanto sarà una costante anche l'offset tra l'indirizzo dell'array e l'indirizzo dell'RdA

- forma fissata al momento dell'elaborazione della dichiarazione
  In questo caso l'intervallo dell'indice dipende dal valore di una variabile, quindi la forma sarà nota solo nel momento in cui il controllo raggiunge la dichiarazione dell'array.
  Anche in questo caso l'array sarà allocato nell'RdA del blocco in cui compare la definizione, però il compilatore non conoscerà l'offset.
  Per questo motivo l'RdA viene diviso in due parti, una per i dati di lunghezza fissa (a cui si accede normalmente per offset), e un'altra per i dati a lunghezza variabile (a cui si accede tramite un descrittore contenuto nei dati a lunghezza fissa)
- forma dinamica
  In questo caso un array può cambiare forma dopo la sua creazione, e pertanto gli array dinamici devono essere allocati sullo heap, mentre il puntatore all'inizio dell'array rimane memorizzato nella parte di lunghezza fissa.

Il descrittore di un array di forma non nota (caso 2 e 3) viene anche chiamata dope vector, e viene allocato nella parte di lunghezza fissa di un RdA e contiene:

- un puntatore alla prima locazione di memoria nella quale è memorizzato l'array
- tutte le informazioni dinamiche necessarie al calcolo della quantità, il numero di dimensioni, il valore del limite inferiore di ogni dimensione e l'occupazione di ogni dimensione

Il dope vector è inizializzato al momento della creazione dell'array.
Per accedere ad un elemento dell'array si accede al dope vector (tramite offset rispetto al puntatore RdA), si calcola l'espressione e la si somma all'indirizzo d'inizio dell'array.

Insiemi

Alcuni linguaggi di programmazione permettono di definire tipi insieme, i cui valori sono costituiti dai sottoinsiemi di un altro tipo.
Le operazioni possibili sui valori di tipo insieme sono il test di appartenenza e le usuali operazioni insiemistiche (unione, intersezione, differenza, non sempre il complemento).
Un insieme è rappresentato in memoria come un vettore di bit, di lunghezza pari alla cardinalità del tipo base (quanti elementi ha il tipo base, per i char 128) e ogni bit nella posizione j indica se il j-esimo elemento appartiene o meno all'insieme.
Siccome, al crescere della cardinalità, cresce la lunghezza di questo vettore, i linguaggi limitano spesso i tipi che possono essere usati come tipi base di un insieme.

Puntatori

Alcuni linguaggi permettono di manipolare direttamente gli l-value, e il tipo corrispondente è detto tipo dei puntatori.
Nei linguaggi con modello delle variabili a riferimento (Java) il tipo dei puntatori non è generalmente necessario.
Uno degli usi principali dei puntatori è quello di usarli per costruire valori di tipi ricorsivi (strutture concatenate).
Per ogni tipo "esiste" il suo tipo dei puntatori a esso. Quindi per un tipo T, avremo che $T*$ è il tipo dei puntatori a T.

I puntatori sono indirizzi a locazioni di memoria che contengono valori di tipo T.
Tra tutti i valori che un puntatore può assumere ne esiste uno canonico che non punta ad alcun valore: $null / nil$.
Le operazioni permesse sui valori di tipo puntatore sono la loro creazione, il test di uguaglianza e la dereferenziazione (l'accesso all'oggetto puntato)

In alcuni linguaggi è possibile effettuare alcune operazioni aritmetiche sui puntatori, come incrementarne uno, sottrarre tra loro due puntatori, sommare una quantità arbitraria ad un puntatore… Tutte queste operazioni vanno sotto il nome di aritmetica dei puntatori.
Da notare come, vista l'aritmetica dei puntatori, i linguaggi che li permettono distruggono qualsiasi ambizione alla type safeness, perchè non c'è alcuna garanzia che un puntatore ad un oggetto T punti effettivamente ancora ad un'area di memoria dove è memorizzato un valore di tipo T.

Nel caso in cui si lavori sulla heap, ci può essere o la deallocazione implicita (garbage collector) oppure la deallocazione esplicita.
Il problema con la deallocazione esplicita sono le dangling reference (cioè puntatori diversi da null che puntano ad informazioni non più significative, perché deallocate per esempio).
Nei linguaggi che permettono ai puntatori di riferirsi ad oggetti sulla pila, i dangling reference si possono generare anche senza deallocazione esplicita.
Tuttavia esistono delle tecniche per disinnescare i dangling reference anche in linguaggi dove è permessa la deallocazione esplicita.
Ovviamente i linguaggi che "permettono" il verificarsi di dangling reference non possono essere type safe

Tipi ricorsivi

Un tipo ricorsivo è un tipo composto nel quale un valore del tipo può contenere un valore dello stesso tipo. In molti linguaggi essi sono delle struct nelle quali un campo è dello stesso tipo della struct stessa.
Per concludere la ricorsione, si può assegnare un valore "speciale" (ad esempio null) all'ultima ricorrenza.
Le operazioni permesse sui valori di tipo ricorsivo sono la selezione di una componente e il test di uguaglianza sul valore comune con null.
Solitamente i tipi ricorsivi sono rappresentati come strutture dati sullo heap.

Funzioni

Tutti i linguaggi di alto livello permettono di definire funzioni, ma pochi permettono di denotare il tipo delle funzioni (cioè dargli un nome nel linguaggio). Il tipo di una funzione possiamo indicarlo come $tipo \in input \rightarrow tipo \in output$.
L'operazione principale ammessa su un valore di tipo funzione è l'applicazione, cioè l'invocazione su alcuni argomenti (parametri attuali).

In alcuni linguaggi, il tipo delle funzioni può essere trattato come qualsiasi altro tipo, cioè può venir passato come argomento ad un'altra funzione, e può essere ritornato come risultato di un altra funzione, tali linguaggi appartengono al paradigma funzionale.

Equivalenza

Dopo aver definito i principali tipi dobbiamo discutere delle regole che governano la correttezza di un programma relativamente alla tipizzazione.
La prima classe di queste regole si occupa di definire quando due tipi, formalmente diversi, sono da considerarsi intercambiabili, cioè non distinguibili all'interno di un programma.
Le regole definiscono tra i tipi una relazione di equivalenza: se due tipi sono equivalenti, ogni espressione o valore di un tipo è anche un'espressione o valore dell'altro e viceversa.
I vari linguaggi interpretano una definizione siffatta in due modi diversi (che danno luogo a due distinte regole di equivalenza tra tipi):

- la definizione di tipo può essere **opaca**, nel qual caso dà luogo all'equivalenza **per nome**
- la definizione di tipo può essere **trasparente**, nel qual caso dà luogo all'equivalenza **strutturale**

Equivalenza per nome

Se un linguaggio adotta definizioni di tipo opache, allora ogni nuova definizione introduce un nuovo tipo, diverso da tutti i precedenti, e in questo caso si ha equivalenza per nome
Due tipi sono equivalenti per nome solo se hanno lo stesso nome (cioè un tipo è equivalente solo a se stesso)
In questo caso quindi anche se ridefiniamo dei nuovi tipi con nomi diversi ma uguali strutturalmente (entrambi int) allora il programma li tratta come due tipi distinti.
Questa logica è la scelta che più rispecchia l'intenzione del progettista, se il programmatore ha introdotto due nomi distinti per lo stesso tipo, avrà avuto le sue motivazioni, ed è giusto che il linguaggio mantenga questi tipi separati.

Equivalenza strutturale

Una definizione di tipo è trasparente quando il nome del tipo non è che un'abbreviazione del tipo che viene definito.
In questa logica, due tipi sono equivalenti se hanno la stessa struttura, cioè se sostituendo tutti i nomi con le relative definizioni, si ottengono tipi identici.
L'equivalenza strutturale fra tipi è la (minima) relazione d'equivalenza che soddisfa le seguenti proprietà

- *un nome di tipo è equivalente a se stesso*
- *se un tipo T è introdotto con una definizione $type\ \ T = espressione$, allora T è equivalente a $espressione$*
- *se due tipi sono costruiti applicando lo stesso costruttore di tipo a tipi equivalenti, allora essi sono equivalenti*

L’equivalenza strutturale permette di parlare di tipi equivalenti in generale, e non fa riferimento ad uno specifico programma, in particolare due tipi equivalenti possono sempre essere sostituiti l’uno al posto dell’altro in un qualsiasi contesto senza alterare il significato del programma (trasparenza referenziale).

Compatibilità e conversione

La relazione di compatibilità tra tipi (più debole da quella dell’equivalenza) permette di usare un tipo in un contesto nel quale sarebbe richiesto un altro tipo
Diciamo che il tipo T è compatibile con il tipo S, se un valore di tipo T è ammesso in un qualsiasi contesto in cui sarebbe richiesto un valore di tipo S
In molti linguaggi è la compatibilità e non l’equivalenza a regolare la correttezza di un assegnamento, quella del passaggio dei parametri ecc.
È chiaro che se due tipi sono equivalenti, allora sono anche compatibili l’uno con l’altro; ma in generale la relazione di compatibilità non è simmetrica (esempio $\int$ ¿ e *float*).
La relazione di compatibilità varia moltissimo da linguaggio a linguaggio, elenchiamo alcune delle possibili nozioni che possiamo incontrare, in ordine crescente di generalità; un tipo T può essere compatibile con S quando:

1. T e S sono equivalenti (visto che è una nozione più restrittiva di quella della compatibilità)
2. i valori di T sono un sottoinsieme dei valori di S (è il caso di un tipo intervallo, contenuto nel suo tipo base, e quindi compatibile con esso)
3. tutte le operazioni sui valori di S sono possibili anche sui valori di T (nozione di sottotipo per i linguaggi orientati agli oggetti)
4. i valori di tipo T corrispondono in modo canonico ad alcuni valori di S ($\int$ ¿ compatibile con *float*)
5. i valori di tipo T possono essere fatti corrispondere ad alcuni valori di S (ad esempio troncando/arrotondando un *float* per farlo diventare $\int$ ¿)

Per gestire in modo unificato questa congerie di interpretazioni diverse si introduce la nozione di **conversione di tipo** che può essere fatta in due modi distinti:

- conversione implicita (coercizione, conversione forzata)
  Quando è la macchina astratta a inserire tale conversione
- conversione esplicita (cast)
  Quando la conversione è indicata nel testo del programma

Coercizioni

In presenza di compatibilità tra il tipo T e il tipo S, il linguaggio permette la presenza di un valore T laddove sarebbe richiesto un valore S.
In corrispondenza di ogni situazione del genere, il compilatore (o la macchina astratta) inseriscono una conversione implicita di tipo da T a S che chiamiamo coercizione.
Da un punto di vista implementativo, una coercizione può corrispondere a cose distinte, a seconda della nozione di compatibilità adottata:

1. T è compatibile con S e condividono la rappresentazione in memoria, in questo caso la coercizione rimane a livello sintattico e non genera alcun codice
2. T è compatibile con S perchè esiste un modo canonico di trasformare i valori di T in S, in questo caso la coercizione viene eseguita dalla macchina astratta (da $\int ¿$ a $float$ la macchina astratta inserisce codice per trasformare la rappresentazione in complemento a 2 in quella in virgola mobile)
3. la compatibilità di T con S è stabilita in virtù di una corrispondenza arbitraria tra i valori di T e quelli di S, pure qui la macchina astratta inserisce il codice necessario alla trasformazione.

I linguaggi con forti controlli di tipo tendono ad avere poche coercizioni di tipo 3.

Conversioni esplicite

Le conversioni esplicite sono annotazioni nel linguaggio che specificano che un valore di tipo deve essere convertito in un altro tipo, pure in questo caso ci sono le 3 distinzioni fatte per le coercizioni.
Non tutte le conversioni esplicite sono consentite, ma solo quelle per le quali il linguaggio sa come implementare la conversione.
È evidente che si può sempre inserire una conversione laddove esiste una compatibilità, in questo caso la conversione esplicita è inutile sintatticamente, ma può essere usata per motivi di documentazione.
I linguaggi moderni tendono a favorire i cast rispetto alle coercizioni.

Polimorfismo

Un sistema dei tipi nel quale ogni oggetto del linguaggio ha un unico tipo, si dice **monomorfo**.
Un sistema dei tipi nel quale uno stesso oggetto può avere più di un tipo è detto polimorfo. Per analogia, diremo che un oggetto è polimorfo quando il sistema di tipi gli assegna più di un tipo.
Ad esempio il nome + vale sia per il tipo per $\int ¿$ che per $float$, il valore $null$ ha il tipo puntatore per ogni tipo.
Parliamo di polimorfismo iniziando a distinguere tra due diverse forme:

- polimorfismo ad hoc (overloading)
- polimorfismo universale
    - parametrico
    - di sottotipo (o di inclusione)

Overloading

L'overloading è in realtà polimorfismo solo in apparenza.
Un nome è sovraccaricato (overloaded) quando ad esso corrispondono più oggetti, e viene usata l'informazione del contesto per stabilire quale oggetto è denotato da una specifica istanza di quel nome.
In presenza di coercizioni, può non essere del tutto evidente come un caso di overloading debba essere risolto.

Polimorfismo universale parametrico

Un valore esibisce polimorfismo universale parametrico (o polimorfismo parametrico) quando ha un'infinità di tipi diversi, che si ottengono per istanziazione da un unico schema di tipo generale

Una funzione polimorfa universale dunque, è costituita da un unico codice, ma lavora uniformemente su tutte le istanze del suo tipo generale (ad esempio la sort di un array, o lo swap tra due variabili), questo funziona perché all'interno della funzione non sono utilizzate informazioni o operazioni di tipo.

Il polimorfismo parametrico è presente nei linguaggi di programmazione in due forme notazionalmente distinte, che sono **polimorfismo parametrico esplicito** e **polimorfismo parametrico implicito**

Nel polimorfismo esplicito sono presenti esplicite annotazioni (ad esempio $<T>$) che indicano quali tipi devono essere considerati parametri.

Nel polimorfismo implicito (ad esempio in ML) è il controllore dei tipi che, tramite opportuni controlli, cerca di ottenere per ogni oggetto del linguaggio il suo tipo più generale, dal quale tutti gli altri tipi si possono ottenere per istanziazione dei parametri di tipo

Polimorfismo universale di sottotipo

È presente nei linguaggi orientati agli oggetti ed è una forma più limitata rispetto al parametrico. Anche in questo caso, un oggetto ha un'infinità di tipi diversi, però in questo caso dobbiamo limitarci alle istanziazioni definite da qualche nozione di compatibilità strutturale, e cioè dalla nozione di sottotipo

Un valore esibisce polimorfismo di sottotipo (o limitato) quando ha un'infinità di tipi diversi, che si ottengono per istanziazione da uno schema di tipo generale, sostituendo ad un opportuno parametro i sottotipi di un tipo assegnato.

La situazione di polimorfismo dunque non è generale, ma è limitata ai sottotipi di D.

Evitare i dangling reference

Affrontiamo il problema di quali meccanismi possano essere inclusi in una macchina astratta per impedire dinamicamente la dereferenziazione di un dangling reference (un riferimento "penzolante"), ovvero puntatori a oggetti che non sono più validi

Tombstone

Mediante le tombstone (pietre tombali) una macchina astratta è in grado di segnalare ogni tentativo di dereferenziare un dangling reference.

In pratica in questo metodo, tutte le volte che si definisce un puntatore, viene creata una tombstone e il puntatore riceve l'indirizzo della tombstone (che a sua volta punterà all'oggetto puntato). Quando un puntatore viene assegnato ad un altro è il contenuto del puntatore a cambiare (e non della tombstone).

Così ogni volta che avviene la deallocazione, l'oggetto si cancella, e la tombstone assume un valore particolare che segnala che i dati a cui si riferiva il puntatore sono "morti".
Le tombstone sono allocate in un'area particolare della memoria (il "cimitero") che può essere gestita in modo più efficiente dello heap.
Il meccanismo delle tombstone però ha molti difetti:

- Tempo e risorse alla creazione della tombstone e al suo controllo
- Doppio accesso indiretto
- Spazio occupato in più
- Rimangono sempre allocate (si può applicare un garbage collector ma costa ancora di più in termini di tempo del meccanismo)

Lucchetti e chiavi

È un meccanismo che non soffre del problema dell'accumulo delle tombstone.
Ogni volta che viene creato un oggetto sullo heap, viene associato all'oggetto anche un "lucchetto". Il puntatore invece è costituito da una coppia: indirizzo e "chiave", la quale viene inizializzata al valore del lucchetto corrispondente all'oggetto puntato.
Così ogni volta che si esegue la dereferenziazione si controlla se la coppia chiave-lucchetto coincide, altrimenti si segnala un errore. E ogni volta che c'è una deallocazione, il lucchetto viene annullato.
Può capitare che l'area di memoria precedentemente usata come lucchetto venga riutilizzata, ma è assai improbabile che coincida perfettamente con la chiave.
Visto che hanno bisogno di una parola aggiuntiva per ogni puntatore, sono più costosi delle tombstone, però qui si ha il vantaggio che vengono deallocate insieme al resto.

Garbage collection

Nei linguaggi senza deallocazione esplicita, si rende necessario dotare la macchina astratta di un meccanismo con il quale si possa automaticamente recuperare la memoria allocata sullo heap e non più utilizzata. Questo meccanismo si chiama garbage collector (spazzino).
Il suo funzionamento si compone in due fasi:

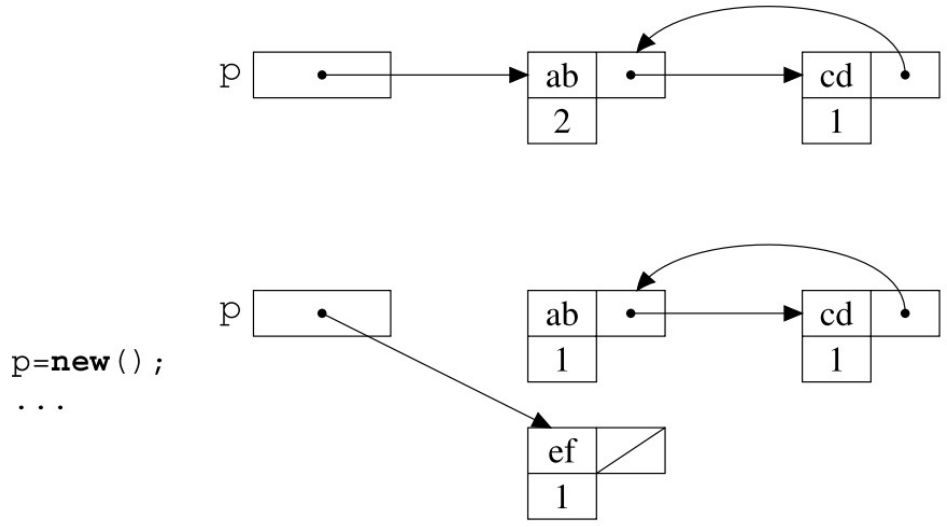
1. distinguere gli oggetti vivi da quelli non più utilizzati (garbage detection)
2. recuperare gli oggetti non più utilizzati, così che il programma possa riutilizzare quegli spazi di memoria

Per motivi di efficienza però, non tutti gli oggetti che non saranno più usati sono in realtà scoperti come tali.
Possiamo classificare i garbage collector classici a seconda di come determinano gli oggetti non più in uso:

- basati sui contatori dei riferimenti
- su marcatura
- su copia

Contatore dei riferimenti

La risposta più semplice che possiamo dare alla domanda relativa a quando un oggetto non è più utilizzato è: quando non vi sono più puntatori verso di esso. Su questo si basa questa tecnica e costituisce il modo più elementare per realizzare un garbage collector.
Per ogni creazione sullo heap di un oggetto, viene allocato anche un intero (il contatore dei riferimenti) e la macchina astratta si incarica di far sì che per ogni oggetto, tale contatore contenga il numero di puntatori attivi a quell'oggetto.
Un chiaro vantaggio di questa modalità è che controllo e recupero sono mescolati con il funzionamento normale del programma.
Il difetto maggiore sta nella difficoltà di deallocare tutte le strutture circolari



I contatori poi sono inefficienti, in quanto se un programma chiama spesso funzioni a cui passa i puntatori, i contatori verranno modificati per pochissimo tempo (inutilmente) sprecando tempo e processore.

Mark and sweep

Questa tecnica si compone appunto di due fasi:

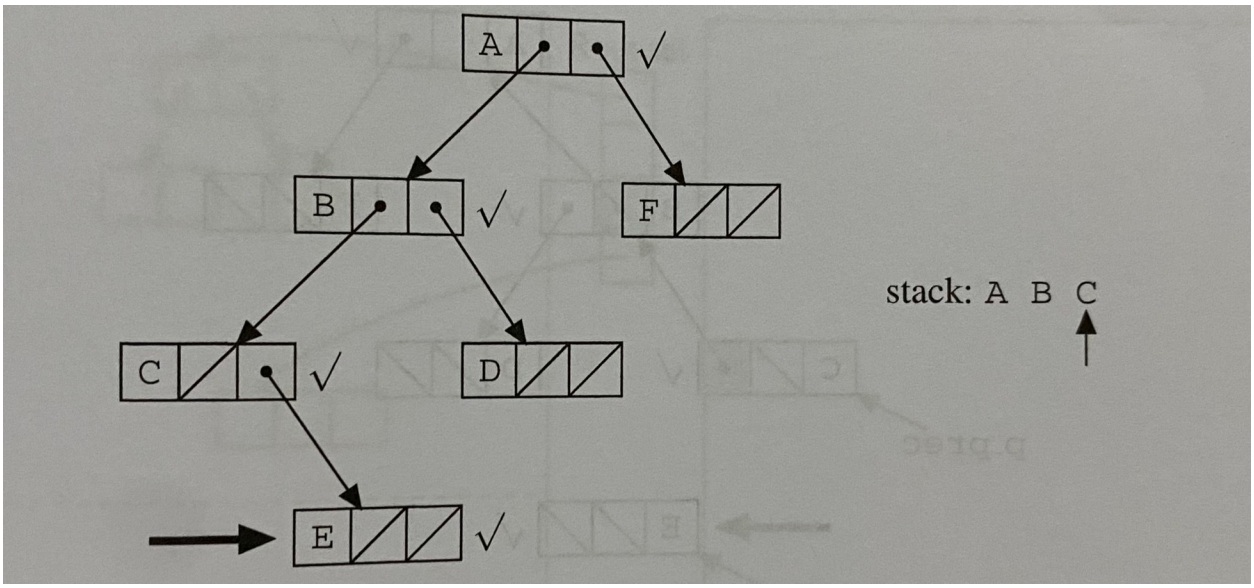
1. mark
   Si "marca" ogni oggetto presente nello heap come "inutilizzato", successivamente partendo dai puntatori attivi presenti sulla pila, si marca come "in uso" ogni oggetto raggiunto da questi puntatori
2. sweep
   Lo heap viene spazzato (swept), quindi tutti gli oggetti marcati come "inutilizzato" vengono restituiti alla lista libera dello heap

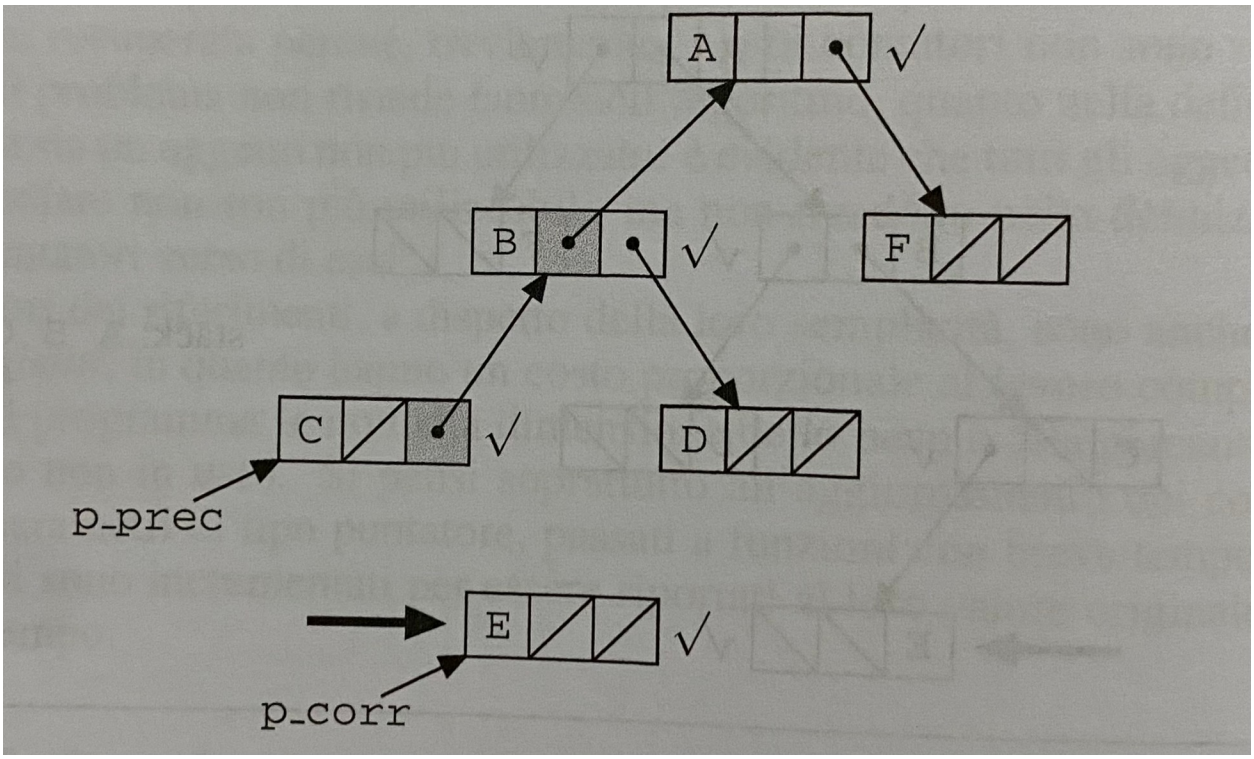
In questo caso il garbage collector non è incrementale, ma verrà invocato solo quando la memoria starà per esaurirsi.

Ha 3 difetti principali: causa frammentazione esterna, è inefficiente (richiede tempo proporzionale alla dimensione totale dello heap), fa sì che oggetti di età molto diverse siano contigui.

Intermezzo: rovesciare i puntatori

Senza qualche precauzione, ogni tecnica di marcatura corre il rischio di essere completamente inutilizzabile. La fase di marcatura comporta la visita ricorsiva di un grafo, che necessita in modo essenziale di una pila per memorizzare i punti di ritorno.
Pertanto occorre usare una tecnica chiamata **rovesciamento dei puntatori** (pointer reversal) e consiste nel rovesciare i puntatori durante la “discesa”, così la risalita si può fare senza aver bisogno di una pila d’appoggio.





Mark and compact

Per ovviare alla frammentazione causata dalla tecnica di mark and sweep possiamo modificare la fase di sweep e convertirla in una fase di compattamento, quindi spostare e rendere contigui gli oggetti vivi e creare un unico blocco contiguo di memoria libera.
C'è maggiore costo in termini di operazioni ma c'è un vantaggio in termini di frammentazione e memoria.

Copia

Questa tecnica consiste nel dividere in due la heap e usarne una metà alla volta. Quando viene invocato il garbage collector, si scorrono tutti i puntatori, e si copiano nell'altra metà gli oggetti (cambiando il valore dei puntatori) e si ripete questa operazione sull'altra metà la volta successiva.

Astrarre sui dati

Tipi di dato astratti

L'introduzione di nuovi tipi di dati, tramite i meccanismi discussi nel capitolo precedente, non permette all'utente di definire tipi con lo stesso grado di astrazione goduto dai tipi predefiniti del linguaggio.
Ad esempio creando una pila di interi tramite struct, nessuno vieta all'utente di andare a modificare direttamente il dato che indica l'indice dell'ultimo elemento inserito, e questo può

causare problemi. Quindi non è possibile nascondere l'implementazione come succede per i tipi di dati predefiniti dal linguaggio.

Per questo motivo certi linguaggi permettono di definire delle astrazioni sui dati, che si comportano come i tipi predefiniti (per quanto riguarda l'inaccessibilità della rappresentazione). Questo meccanismo è detto **tipo di dato astratto** (o ADT) ed è caratterizzato dai seguenti aspetti principali:

1. Un nome per il tipo
2. Un'implementazione (o rappresentazione) per tale tipo
3. Un insieme di nomi di operazioni per la manipolazione dei valori di quel tipo con i loro tipi
4. Per ogni operazione, un'implementazione che usi la rappresentazione fornita al punto 2
5. Una capsula di sicurezza che separi i nomi del tipo e delle operazioni dalle loro realizzazioni

La chiave di questo meccanismo è che all'interno della dichiarazione di un tipo di dato astratto, il tipo di dato astratto è un sinonimo della sua rappresentazione concreta, mentre fuori di essa, non c'è più alcuna relazione tra il tipo di dato astratto e il suo tipo concreto. Di conseguenza il tipo di dato astratto può essere manipolato solamente utilizzando le operazioni definite per lui.

Un ADT è una capsula opaca: dall'esterno è visibile solo il nome del tipo e il nome e il tipo delle operazioni; al suo interno invece sono visibili anche le implementazioni del tipo e delle operazioni. L'accesso all'interno è sempre mediato dalla capsula.
Questa superficie esterna della capsula si chiama **segnatura** (signature) oppure **interfaccia dell'ADT**, al suo interno invece si trova l'**implementazione**
I tipi di dati astratti si comportano come i tipi predefiniti: è possibile dichiarare variabili di un tipo astratto e usare un tipo astratto per definirne un altro.

Nascondere l'informazione

La distinzione tra interfaccia ed implementazione è di grande importanza per le tecniche di sviluppo del software, perché permette di separare l'uso di un componente dalla sua definizione.
Nell'astrazione di un dato non viene nascosto solo il "come" una certa operazione è realizzata ma anche le modalità di rappresentazione dei dati, questo fenomeno viene indicato come **occultamento dell'informazione** (information hiding). Una conseguenza importante è il fatto che è possibile sostituire l'implementazione di un ADT con un'altra, mantenendo immutata l'interfaccia.
Viene chiamata **specifica** la descrizione della semantica delle operazioni di un ADT, ed è la cosa che resta invariata quando si modifica l'implementazione. Questa proprietà si chiama **principio di indipendenza dalla rappresentazione**.

La specifica di un tipo di dato astratto può essere data in molti modi diversi, che vanno dal linguaggio naturale, a schemi formali, a linguaggi completamente formalizzati.
L'ADT garantisce che l'implementazione delle operazioni soddisfi la specifica.

Indipendenza della rappresentazione

Proprietà di indipendenza dalla rappresentazione: *Due implementazioni corrette di una stessa specifica di un ADT sono osservabilmente indistinguibili da parte dei clienti di quel tipo*.
Quindi se un tipo gode dell'indipendenza, è possibile modificare la sua implementazione con una equivalente senza che ciò provochi nei clienti la comparsa di nuovi errori.

Moduli

I tipi di dato astratto sono meccanismi della "programmazione in piccolo", sono pensati per incapsulare **un** tipo con le relative operazioni.
È molto più comune invece che un'astrazione sia composta da più tipi (o strutture dati) tra loro correlati.
I meccanismi linguistici che realizzano questo tipo di incapsulamento vengono solitamente chiamati **moduli** o **package** e appartengono alla "programmazione in grande", cioè alla realizzazione di un sistema complesso mediante la composizione e assemblaggio di componenti più semplici.
Il meccanismo dei moduli permette di partizionare un programma in parti distinte, ciascuna delle quali è dotata sia di dati che di operazioni; un modulo raggruppa più dichiarazioni e definisce delle regole di visibilità per quelle dichiarazioni, mediante le quali si può realizzare una forma di incapsulamento e di occultamento dell'informazione.

Da un punto di vista dei principi non cambia nulla dagli ADT, se non per il fatto che è possibile definire più tipi contemporaneamente.
Da un punto di vista pragmatico, invece, il meccanismo dei moduli fornisce molta flessibilità in più, sia nella definizione del grado di permeabilità della capsula, sia nella possibilità di definire moduli generici, cioè polimorfi. Solitamente i costrutti relativi ai moduli sono congiunti con un meccanismo di compilazione separata dei moduli stessi.

Come un ADT, un modulo ha una parte pubblica e una parte privata. Un modulo cliente può usare la parte pubblica di un altro modulo **importandolo**.

Il paradigma logico

Deduzione come computazione

L'attività di programmazione può essere descritta con la seguente "equazione":

$$Algoritmo = Logica + Controllo$$

Quindi la specifica di un algoritmo, in termini di linguaggio di programmazione, può essere separato in due parti:

- La logica
  Ovvero **cosa** deve essere fatto
- Il controllo
  Ovvero **come** deve essere fatto

A differenza di un linguaggio imperativo, dove il programmatore deve tener conto di entrambe le componenti, in un linguaggio a programmazione **logica**, il programmatore ha il solo compito di implementare la parte logica, e tutto quello riguardante il controllo viene demandato alla macchina astratta, che tramite un meccanismo computazionale, basato su una particolare regola di deduzione (la risoluzione), definisce la sequenza di operazioni necessarie ad arrivare al risultato finale.

Al giorno d'oggi, tutti i linguaggi costruiti sul paradigma logico, permettono anche l'uso di costrutti che riguardano anche aspetti di specifica del controllo.

Questi linguaggi sono molto potenti, in quanto permettono di esprimere in modo compatto e relativamente semplice programmi che risolvono problemi anche molto complessi, ovviamente però c'è da pagare in termini di efficienza.
Per trovare la soluzione viene usato un meccanismo di **backtracking**, ovvero ogni qualvolta che la computazione arriva ad un punto nel quale non può procedere, viene "disfatta" la computazione effettuata fino a tornare ad un punto di scelta fino a che non c'è più una scelta oppure si trova la soluzione.
Si capisce subito che questo processo "per tentativi" può avere complessità esponenziale

Sintassi

I programmi logici sono insiemi di formule logiche di una particolare forma.
Ci occuperemo della logica di primo ordine (o anche calcolo dei predicati), ovvero dove si usano dei simboli per esprimere proprietà di elementi che fanno parte di un dominio (secondo ordine: insiemi e funzioni; terzo ordine: insiemi di funzioni; etc).

Il linguaggio della logica del prim'ordine

Per poter parlare del calcolo dei predicati, dobbiamo definirne il linguaggio.
Un **linguaggio del prim'ordine** consiste di tre componenti:

1. Un **alfabeto**
2. I **termini** definiti su tale alfabeto
3. Le **formule ben formate** definite su tale alfabeto

Alfabeto

L'alfabeto è un insieme di simboli, che possiamo considerare partizionato in due sottoinsiemi disgiunti: l'insieme dei **simboli logici** (comuni a tutti i linguaggi del prim'ordine) e l'insieme dei **simboli non logici** (specifici di un qualche dominio di interesse).
L'insieme dei simboli logici contiene i seguenti elementi:

- I connettivi logici
    - $\wedge$ congiunzione
    - $\vee$ disgiunzione
    - $\neg$ negazione
    - $\rightarrow$ implicazione

    - $\leftrightarrow$ doppia implicazione
- Le costanti proporzionali
    - *true*
    - *false*
- I quantificatori
    - $\exists$ esiste
    - $\forall$ per ogni
- Simboli di interpunzione
    - $()$ parentesi
    - , virgola
- Un insieme finito V di variabili, indicate con $X, Y, Z, \ldots$

I simboli non logici sono definiti da una **segnatura con predicati** $\left(\sum_{\square}^{\square}, \prod_{\square}^{\square} \square\right)$.

Si tratta di una coppia nella quale il primo elemento $\sum_{\square}^{\square} \square$ è la **segnatura delle funzioni**, ognuno con la propria arietà (il numero di argomenti di una funzione/relazione) e il secondo elemento $\prod_{\square}^{\square} \square$ è la **segnatura dei predicati**, ognuno con la propria arietà.

La differenza è che i primi devono essere interpretati come funzioni, i secondi invece come relazioni.

Termini

Un'espressione aritmetica è in sostanza un termine ottenuto applicando degli operatori (aritmetici) a degli operandi, e questo ragionamento si può applicare a stringhe, alberi binari, liste ecc.

Quindi, nel caso più semplice, un termine è ottenuto applicando un simbolo di funzione a variabili e costanti in modo tale da rispettare l'arietà.

I termini sulla segnatura $\sum_{\square}^{\square} \square$ *(e sull'insieme di variabili V) sono definiti induttivamente come segue:*

- *Una variabile (in V) è un termine*
- *Se* f *(in* $\sum_{\square}^{\square} \square$*) è un simbolo di funzione di arietà n e* $t_1, \ldots, t_n$ *sono termini, allora* $f(t_1, \ldots, t_n)$ *è un termine.*

Notiamo che le costanti possono essere viste come termini, e corrispondono a funzioni di arietà 0. Per convenzione omettiamo le parentesi, quindi invece di $a(), b()$ avremo a, b.

I termini senza variabili sono detti termini **ground**.

Notiamo che nei termini non compaiono predicati, essi compariranno nelle formule (per esprimere proprietà dei termini)

Formule

Le **formule ben formate** ci permettono di esprimere proprietà dei termini che poi, da un punto di vista semantico, sono proprietà di un particolare dominio di interesse.
Con le formula ad esempio si può definire la proprietà di transitività del simbolo $>$ maggiore:

$$>(X,Y) \ \wedge \ >(Y,Z) \rightarrow >(X,Z)$$

Le formule (ben formate) del linguaggio sulla segnatura con termini $\left(\sum_{\square}^{\square}, \prod_{\square}^{\square} \square\right)$ *sono definite induttivamente come segue:*

1. *Se* $t_1,\ldots,t_n$ *sono termini sulla segnatura* $\sum_{\square}^{\square} \square$ *e* $p \in \prod_{\square}^{\square} \square$ *è un simbolo di predicato di arietà n, allora* $p(t_1,\ldots,t_n)$ *è una formula*
2. *true e false sono formule*
3. *Se* F *e* G *sono formule, allora* $\neg F, (F \wedge G), (F \vee G), (F \rightarrow G)$ *e* $(F \leftrightarrow G)$ *sono formule*
4. *Se* F *è una formula e* X *è una variabile, allora* $\forall X.F$ *e* $\exists X.F$ *sono formule*

I programmi logici

Una formula del prim'ordine può avere una struttura molto complessa con una conseguente difficoltà di determinare una dimostrazione per essa.
Per questo sono state identificate particolari classi di formule, dette **clausole**, che si prestano ad essere manipolate più efficientemente, tramite una particolare regola di dimostrazione dette **risoluzione**.

Siano $H, A_1,\ldots,A_n$ *formule atomiche. Una clausola definita è una formula della forma*

$$H \ :\!- A_1,\ldots,A_n.$$

Se $n=0$ *la clausola è detta unitaria, o fatto, ed il simbolo* :- *è omesso (ma non il punto terminale). Un programma logico è un insieme di clausole, mentre un programma PROLOG puro è una sequenza di clausole. Una query (o goal) è una sequenza di atomi* $A_1,\ldots,A_n$.
Il simbolo :- rappresenta un implicazione rovesciata ($\leftarrow$), la virgola invece, dal punto di vista logico, è da intendersi come congiunzione, il punto è importante per segnalare quando termina la clausola.
La parte a sinistra di :- è detta **testa** della clausola, invece quella a destra è detta **corpo**.
Un fatto è una clausola con corpo vuoto.
L'insieme delle clausole che contengono nella testa il simbolo di predicato p è detto la **definizione** di p. Le variabili che occorrono nel corpo di una clausola sono dette **variabili locali**.

Teoria dell'unificazione

Il meccanismo fondamentale di computazione nella programmazione logica è la soluzione di equazioni fra termini mediante il processo di unificazione, in questo processo vengono calcolate le sostituzioni che legano delle variabili a dei termini.

La variabile logica

La nozione di variabile è diversa da quella vista precedentemente. La variabile logica costituisce sempre un'incognita che può assumere valori di un insieme prefissato, però ci sono 3 importanti differenze tra questa nozione e la variabile modificabile dei linguaggi imperativi:

1. La variabile logica è legata una sola volta, cioè se una variabile è legata ad un termine, questo non può essere distrutto (può solo venir modificato il termine).
   Quindi se leghiamo la variabile X alla costante a, non possiamo sostituire il legame con un altro che lega la variabile X con la costante b, però si può modificare il valore della variabile.
2. Il valore di una variabile logica può essere definito parzialmente (o indefinito) per poi essere ulteriormente specificato in seguito (ad esempio se leghiamo la variabile X al termine $f(Y,Z)$ e successivamente creiamo i legami di Y e Z andremo a cambiare il valore di X)
3. Abbiamo una natura bidirezionale dei legami nel caso delle variabili logiche.
   Quindi se X è legata al termine $f(Y)$ e successivamente leghiamo X a $f(a)$, l'effetto che produciamo è quello di legare la variabile Y con a.
   Notiamo che questo non contraddice il primo punto, in quanto il legame non viene distrutto ma viene solamente "specificato"

Sostituzione

Il legame tra variabili e termini è realizzato dalla sostituzione, che appunto permette di sostituire un termine al posto di una variabile.
Un sostituzione (ϑ, σ, ρ) può essere definita così:
Una sostituzione è una funzione da variabili a termini tale che il numero di variabili che non sono mappate in sé stesse è finito.
Indichiamo una sostituzione ϑ usando la notazione

$$\vartheta = \{X_1/t_1, \ldots, X_n/t_n\}$$

dove $X_1, \ldots, X_n$ sono variabili diverse, $t_1, \ldots, t_n$ sono termini e dove assumiamo che t_i sia diverso da X_i, per $i = 1, \ldots, n$.
Una coppia X_i/t_i è detta legame (binding), nel caso in cui tutti i termini $t_1, \ldots, t_n$ siano ground, allora ϑ è detta sostituzione ground.
ε denota la sostituzione vuota.
Per ϑ definita come prima, definiamo dominio, codominio e le variabili della sostituzione nel seguente modo:

$$Dominio(\vartheta) = \{X_1, \ldots, X_n\}$$

$$Codominio(\vartheta) = \{Y \mid Y \text{ è una variabile} \in t_i, \text{ per qualche } t_i,\ 1 \le i \le n\}$$

Una sostituzione può essere applicata ad un termine, o in generale a una qualsiasi espressione sintattica, per modificare il valore delle variabili presenti nel dominio.

Esiste un tipo di sostituzione che permette semplicemente di cambiare il nome alle variabili.

Una sostituzione ρ è una ridenominazione se esiste la sua sostituzione inversa ρ^{-1} tale che $\rho\rho^{-1}=\rho^{-1}\rho=\varepsilon$

Esiste infine un preordine $\leq$ sulle sostituzioni, dove $\vartheta\leq\sigma$ è letto come ϑ è più generale di σ, e questo è vero se e solo se esiste una sostituzione γ tale che $\vartheta\gamma=\sigma$

L'unificatore più generale

Il meccanismo di computazione di base del paradigma logico è la valutazione di equazioni, dove il simbolo = è un simbolo di predicato interpretato come uguaglianza sintattica di tutti i termini ground.
Se in un programma logico scriviamo $X=a$ intendiamo dire che la variabile X deve essere legata alla costante a. La sostituzione $\{X/a\}$ è una soluzione, in quanto $a=a$ è soddisfatta.
La differenza coi linguaggi imperativi è che è una operazione di assegnamento, infatti scrivendo $a=X$ il risultato è lo stesso.
Infatti $3=2+1$, trattandosi di un'uguaglianza sintattica sui termini ground, dice che non è risolvibile, o che è falsa, in quanto la costante 3 è diversa dal termine $2+1$ (essendo termini ground non esiste un modo per renderli sintatticamente uguali.
Notiamo che in generale l'equazione $X=t$ non è risolvibile se t contiene la variabile X (ed è diverso da X).
Quindi $f(X)=f(g(X))$ non è risolvibile, invece $f(X)=f(g(Y))$ lo è, in quanto **una** soluzione è la sostituzione $\vartheta=\{X/g(Y)\}$. Detto in termini più formali, la soluzione ϑ unifica i due termini dell'equazione, pertanto è detta **unificatore**.
Essendo che esistono infinite sostituzioni che unificano i due termini, ad esempio $\{X/g(a),Y/a\}$, $\{X/g(f(Z)),Y/f(Z)\}$, *etc*., diciamo quindi che $\vartheta=\{X/g(Y)\}$ è l'**unificatore più generale** o anche m.g.u (most general unifier) di X e $g(Y)$.
Un altro particolare importante è che il processo di soluzione non ha una specifica "direzione", ovvero si può sostituire il termine a destra, oppure quello a sinistra, oppure entrambi.
Dato un insieme di equazioni $E=\{s_1=t_1,\ldots,s_n=t_n\}$ dove $s_1,\ldots,s_n$ e $t_1,\ldots,t_n$ sono termini, la sostituzione ϑ è un unificatore per E se le sequenze $(s_1,\ldots,s_n)\vartheta$ e $(t_1,\ldots,t_n)\vartheta$ sono sintatticamente identiche.
Un unificatore di E è detto unificatore più generale (m.g.u.) se più più generale di ogni altro unificatore di E, ossia se per ogni altro unificatore σ di E vale che σ è eguale a $\vartheta\tau$ per un'opportuna sostituzione τ

Un algoritmo di unificazione

Un risultato importante dimostra che è decidibile il problema di verificare se un insieme di equazioni fra termini è unificabile. La dimostrazione è costruttiva (fornisce un vero e proprio algoritmo di unificazione) e per ogni insieme di equazioni produce il loro m.g.u. se l'insieme è unificabile.
Inoltre dimostra che un m.g.u. è unico a meno di renaming.

Dato un insieme di equazioni

$$E = \{ s_1 = t_1, \ldots, s_n = t_n \}$$

L'algoritmo o produce un fallimento oppure un insieme di equazioni in forma risolta del tipo

$$E = \{ X_1 = r_1, \ldots, X_m = r_m \}$$

L'algoritmo per arrivare al risultato finale compie i seguenti passi:

1. $f(l_1, \ldots, l_k) = f(m_1, \ldots, m_k)$: elimina questa equazione dall'insieme E e aggiunge le equazioni del tipo $l_1 = m_1, \ldots, l_k = m_k$
2. $f(l_1, \ldots, l_k) = g(m_1, \ldots, m_h)$: se f è diverso da g termina con fallimento
3. $X = X$: elimina questa equazione dall'insieme E
4. $X = t$: se t non contiene la variabile X e tale variabile appare in un'altra equazione dell'insieme E, applica la sostituzione $\{X / t\}$ a tutte le altre equazioni dell'insieme E
5. $X = t$: se t contiene la variabile X termina con fallimento
6. $t = X$: se t non è una variabile elimina questa equazione dall'insieme E e aggiunge ad E l'equazione $X = t$

Il modello computazionale

Il paradigma logico, realizzando l'idea di "computazione come deduzione", ha un modello computazionale sostanzialmente diverso dagli altri paradigmi. Possiamo individuare le seguenti differenze rispetto agli altri paradigmi:

1. Gli unici valori possibili (almeno nel modello puro) sono i termini su una data segnatura
2. I programmi possono avere una lettura dichiarativa, interamente logica, o una lettura procedurale di tipo più operazionale
3. La computazione avviene istanziando le variabili che appaiono nei termini (e quindi nei goal) con altri termini, usando il meccanismo di unificazione
4. Il controllo, interamente gestito dalla macchina astratta, è basato sul meccanismo del backtracking automatico

L'universo di Herbrand

L'insieme di tutti i possibili termini su una data segnatura è detto universo di Herbrand, e costituisce il dominio sul quale viene effettuata la computazione. Vi sono alcune particolarità:

- L'alfabeto dei simboli non logici può variare
- Nessun significato predefinito è associati ai simboli (non logici) dell'alfabeto (ad eccezione dei predicati built-in)
- Nessun sistema di tipi è presente nei linguaggi logici, l'unico tipo è quello dei termini.

Dal punto di vista teorico il fatto che non vi siano tipi non è limitativo, ma permette di esprimere in modo elegante e semplice la semantica formale dei programmi logici.
Dal punto di vista pratico invece la mancanza di tipi è un problema, infatti alcuni linguaggi più recenti hanno introdotto alcuni tipi elementari, con le loro relative operazioni.

Interpretazione dichiarativa e procedurale

Come già accennato, un programma logico può avere una interpretazione **dichiarativa** e una interpretazione **procedurale**.

Dal punto di vista dichiarativo, una clausola H :- $A_1,\ldots,A_n$. è una formula, che semplicemente esprime che se $A_1,\ldots,A_n$ sono veri, allora è vero anche H.
Una query (o goal) è anch'essa una formula, per la quale vogliamo dimostrare che, se opportunamente istanziata, è una conseguenza logica del programma, ovvero vale in tutte le interpretazioni nelle quali vale il programma.

L'interpretazione procedurale invece permette di leggere una clausola H :- $A_1,\ldots,A_n$. come "per dimostrare H devi dimostrare $A_1,\ldots,A_n$". In questo senso possiamo vedere un predicato come un nome di una procedura.

Precisi teoremi consentono di dimostrare che i due approcci sono equivalenti

La chiamata di procedura

Consideriamo una definizione semplificata di clausola, assumendo che nella testa tutti gli argomenti del predicato siano variabili distinte. Una generica clausola di questo tipo ha dunque la forma:

$$p(X_1,\ldots,X_n) \text{ :- } A_1,\ldots,A_m.$$

Questa può essere vista come la dichiarazione della procedura p con n parametri formali.
Quindi scrivendo $p(t_1,\ldots,t_n)$ eseguiamo la chiamata alla procedura con $t_1,\ldots,t_n$ parametri attuali.
All'interno della procedura avviene una sostituzione simile a quella per nome (quindi sostituendo ogni ricorrenza del parametro formale con quello attuale).
Quindi la valutazione della chiamata $p(t_1,\ldots,t_n)$, causa la valutazione delle m chiamate di procedura $(A_1,\ldots,A_m)\vartheta$ presenti nel corpo di p, istanziate dalla sostituzione $(A_1,\ldots,A_m)\{X_1/t_1,\ldots,X_n/t_n\}$.
La soluzione sarà la sostituzione che associa alle variabili presenti nella chiamata iniziale $(X_1,\ldots,X_n)$ i valori calcolati nel corso della computazione. Questa soluzione è detta **risposta calcolata** per il goal iniziale nel programma dato

Valutazione di un goal non atomico

Da ora chiameremo **goal atomico** una chiamata di procedura e **goal** una sequenza di chiamate.
Nel caso di un goal non atomico, il meccanismo computazionale è analogo, però dobbiamo usare una opportuna **regola di selezione** (anche se è importante notare che l'ordine non è influente per il risultato, che sarà sempre lo stesso)

Teste con termini generici

Abbiamo assunto che le variabili presenti nelle teste delle clausole fossero tutte distinte, in realtà si può tranquillamente riutilizzare una variabile.

Controllo: il non determinismo

Quando dobbiamo valutare un goal, abbiamo due libertà di scelta:

- La selezione dell'atomo da valutare
- La scelta della clausola da applicare

Per il primo abbiamo appena detto che qualsiasi regola va bene, tanto il risultato rimane invariato.
Per il secondo invece la questione è più delicata, potremmo fissare un ordine fisso (ad esempio dall'alto verso il basso) ma questo potrebbe portare ad un loop infinito (ad esempio se la prima clausola è $p(X) :- p(X)$.
Quindi dobbiamo sceglierne una in modo non-deterministico, cioè senza applicare nessuna regola, così facendo riusciamo a considerare tutte le possibili scelte delle clausole e quindi tutti i possibili risultati. Quindi il risultato della valutazione di un goal G è un insieme di risposte calcolate

Il backtracking in PROLOG

Quando si passa dal modello teorico a quello pratico, cioè quando si implementa, il non-determinismo deve per forza diventare in qualche modo un determinismo. Esistono vari modi per l'implementazione ed, in generale, nessuna di questa causa la perdita di soluzioni.
In PROLOG per motivi di semplicità e efficienze, le clausole vengono usate secondo l'ordine testuale in cui appaiono nel programma, anche se abbiamo visto che è una strategia incompleta, in quanto non permette di trovare tutte le possibili soluzioni.
Tuttavia questo limite è aggirabile dal programmatore, in quanto può riordinare le clausole in modo da avere per prime quelle "terminali" e poi quelle "induttive".
Questo però elimina la caratteristica della dichiaratività del linguaggio, in quanto il programmatore implementa una logica di controllo.

In generale, quando si arriva ad un fallimento, la macchina astratta PROLOG fa "backtracking", ovvero torna al punto di scelta precedente, distruggendo tutti i legami eseguiti da dopo quel punto, e prende un'altra scelta (se le scelte sono già state tutte prese si ritorna indietro al punto di scelta precedente). Notiamo che questa soluzione può essere molto dispendiosa.

Alcuni esempi

Faremo riferimenti al linguaggio PROLOG
La notazione $[h \mid t]$ è usata per indicare la lista che ha come testa h e come coda t.
Ricordiamo che la tesa è il primo elemento, invece la coda tutti i successivi.
La lista vuota è denotata da $[]$ mentre la scrittura $[a, b, c]$ è l'abbreviazione per $[a \mid [b \mid [c \mid []]]]$.
Notiamo che in PROLOG non esiste il tipo lista, quindi possiamo anche scrivere $[a \mid f(a)]$

Estensioni

In questo paragrafo accenneremo brevemente ad alcune delle numerose estensioni del formalismo puro.

PROLOG

PROLOG è un linguaggio molto più ricco di quello che potrebbe apparire da quanto detto finora. Ricordiamo che PROLOG adotta delle precise regole per la selezione dell'atomo da eseguire (da sinistra a destra) e per la scelta delle clausole da usare (dall'alto in basso), ma ha altre differenze rispetto al formalismo teorico:

Aritmetica

Siccome un linguaggio reale non può permettersi di usare un aritmetica completamente simbolica, in PROLOG esistono gli interi e i reali (in virgola mobile) come strutture dati predefiniti, assieme a vari operatori:

- $+, -, *, e\ //\ (divisione\ \ intera)$
- $<, <=, >, >=, =:=\ (uguale), =¿\ (diverso$
- Un operatore di valutazione detto *is*

Questi simboli, a differenza degli altri (di funzione e di predicato) utilizzano la notazione infissa. Gli operatori di confronto richiedono sempre operando che siano espressioni aritmetiche ground (cioè senza variabili) altrimenti segnalano un errore. Infatti una cosa del tipo $X\ =:=\ 3+5$ non è possibile come uguaglianza. Abbiamo anche visto che non è possibile $X=3+5$ come assegnamento, infatti l'operatore *is* fa proprio questo: $X\ \ is\ \ 3+5.$
Infatti *is* **unifica** con il **valore** dell'espressione aritmetica ground (se non è ground abbiamo un errore)

Cut

PROLOG offre vari costrutti per modificare il normale flusso del controllo, uno dei più significativi è il **cut**. È un predicato senza argomenti indicato da un punto esclamativo.
È usato per eliminare delle possibili alternative nel processo di valutazione, quando non vogliamo che altre chiamate a quel goal diano altri risultati.

Disgiunzione

Se vogliamo esprimere la disgiunzione di due goal $G1$ e $G2$, ovvero che almeno uno dei due deve avere successo, possiamo usare due clausole della forma:
$p(X):-G1.$
$p(X):-G2.$
Oppure si può semplicemente usare il predicato di disgiunzione built-in "*;*"

If-then-else

Il costrutto in PROLOG è offerto come built-in, con la sintassi $B\ \ ->C1;C2$
Oppure si può realizzare usando il cut.

Negazione

Finora abbiamo visto formule atomiche “positive”, ossia non negate.
Per rendere una formula negata basta mettere il *not* davanti
Però non bisogna confonderlo con la negazione logica, in quanto è detta “negazione con fallimento”, ovvero che ritorna false anche quando non termina quella “positiva”.

Programmazione logica e basi di dati

Un insieme di clausole unitarie, possono essere viste a tutte gli effetti come delle query SQL in cui interroghiamo un database.
I predicati definiti da clausole non unitarie invece definiscono delle relazioni in modo implicito (nella terminologia di database si chiamano relazioni virtuali).
Se non facciamo uso di funzioni in un programma logico, stiamo letteralmente costruendo un linguaggio per gestire basi di dati. La definizione di *Datalog* è una versione semplificata della programmazione logica nella quale:

1. non vi sono simboli di funzione
2. si distinguono predicati estensionali, predicati intensionali e predicati di confronto
3. i predicati estensionali non possono comparire nella testa di clausole con corpo non vuoto
4. se una variabile compare nella testa, allora compare anche nel corpo di una clausola
5. un predicato di confronto può comparire solo nel corpo di una clausola; le variabili che compaiono in un predicato di confronto devono comparire anche in un altro atomo del corpo della stessa clausola

I predicati estensionali possono essere definiti solo da fatti.

Programmazione logica con vincoli

Accenniamo brevemente alla programmazione logica con vincoli o CLP (constraint logic programming) la quale unisce il modello computazionale già con meccanismi di soluzione di vincoli.
Un vincolo (constraint) è una particolare formula della logica del prim’ordine che usa solo predicati di significato predefinito. Un esempio di vincolo è $X = t$ che viene interpretato come uguaglianza sintattica sull’universo di Herbrand.
L’idea della programmazione logica con vincoli è quella di rimpiazzare l’universo di Herbrand con un diverso dominio di computazione, simbolico ma anche numerico, adatto alla specifica classe di applicazioni che interessa.
Di conseguenza non avremo più relazioni tra termini ground ma vincoli, e il meccanismo di calcolo non sarà più la soluzione di equazioni tra termini, ma sarà un risolutore di vincoli.
Il vantaggio è il poter integrare col paradigma logico, dei meccanismi di calcolo molto potenti, sviluppati anche in altri contesti

Pregi e difetti del paradigma logico

I linguaggi logici richiedono uno stile di programmazione significativamente diverso e hanno anche un diverso potere espressivo dagli altri tipi di linguaggio.

Ripetiamo che i linguaggi logici, analogamente a quelli funzionali, permettono di esprimere in modo “puramente dichiarativo” soluzioni di problemi anche molto complessi.
Questi aspetti sono ulteriormente rafforzati da tre caratteristiche uniche del paradigma logico:

i. la possibilità di usare in più modi diversi uno stesso programma, trasformando argomenti di input in argomenti di output e viceversa
ii. la possibilità di ottenere come risultato una relazione complessa fra variabili, che esprime, in modo implicito, un'infinità di soluzioni
iii. la possibilità di leggere un programma direttamente come una formula logica

Questi tre aspetti possono essere riassunti nel principio della “computazione come deduzione”, e permettono di esprimere in maniera molto naturale forme di ragionamento tipiche dei problemi che si incontrano nell’ambito dell’intelligenza artificiale e della rappresentazione della conoscenza.
I linguaggi logici possono essere usati con profitto come strumenti per la prototipizzazione rapida di sistemi.

Ovviamente ci sono anche degli aspetti negativi, ad esempio l’inefficienza del meccanismo di backtracking, l’assenza di tipi e moduli che rendono il controllo di correttezza abbastanza difficile.